Apprentissage automatique supervisé

Julien VELCIN

https://velcin.github.io

ECL-3A
Master Informatique
ECL - LIRIS - CNRS

Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

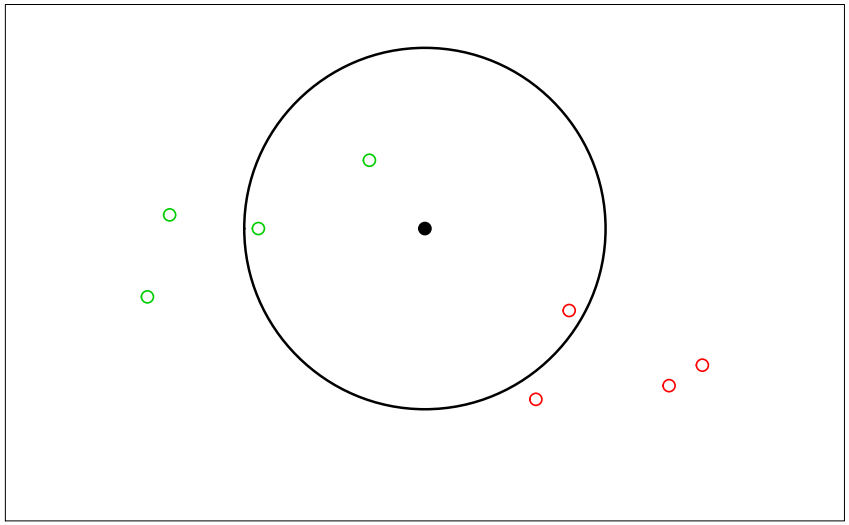
Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

K plus proches voisins (KNN)

Idées :

- On compte parmi les K plus proches voisins d'un points $\mathbf{x}$ à classer le nombre de points n_k de chaque classe ($1 \leq k \leq K$).
- On estime alors la probabilité que $\mathbf{x}$ appartienne à la classe k par :

$$t_k(\mathbf{x}) = \frac{n_k}{K}$$



K plus proches voisins (KNN)

Remarques

- choix de K crucial : CV, AUC, éch. test...
- plus n est grand, plus on peut se permettre de prend un K grand

Plan

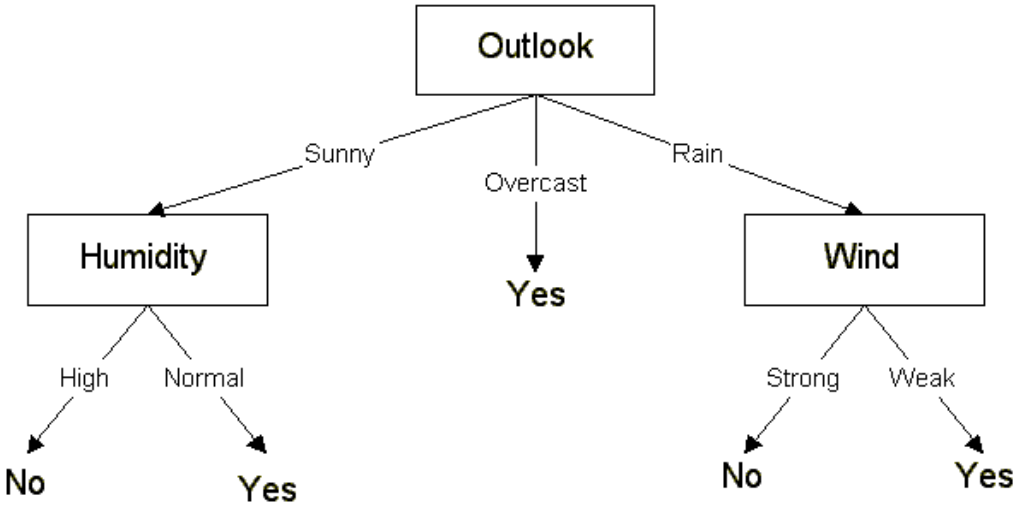
Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

Arbre de décision

Principe

Un arbre où chaque nœud est associé à un test sur une variable et les feuilles spécifient une décision sur la catégorie de l'individu observé.



Exemple des données sur le golf (Quinlan, 1983)

Attributs	Pif	Temp	Humid	Vent	Golf
Valeurs	soleil, couvert, pluie	chaud, bon, frais	normale, haute	vrai, faux	jouer, ne_pas_jouer

1	soleil	chaud	haute	faux	ne_pas...
2	soleil	chaud	haute	vrai	ne_pas...
3	couvert	chaud	haute	faux	jouer
4	pluie	bon	haute	faux	jouer
5	pluie	frais	normale	faux	jouer
6	pluie	frais	normale	vrai	ne_pas...
7	couvert	frais	normale	vrai	jouer
8	soleil	bon	haute	faux	ne_pas...
9	soleil	frais	normale	faux	jouer
10	pluie	bon	normale	faux	jouer
11	soleil	bon	normale	vrai	jouer
12	couvert	bon	haute	vrai	jouer
13	couvert	chaud	normale	faux	jouer
14	pluie	bon	haute	vrai	ne_pas...

classe

Apprendre un arbre

Approches

- générer tous les arbres possibles (combinatoire)
- choisir un biais : exploration ascendante ou descendante, gestion des valeurs numériques, etc.
- approche descendante : choisir une variable à chaque étape

Apprendre un arbre

Approches

- générer tous les arbres possibles (combinatoire)
- choisir un biais : exploration ascendante ou descendante, gestion des valeurs numériques, etc.
- approche descendante : choisir une variable à chaque étape

Algorithme ID3 (Quinlan, 1983)

- premier algorithme proposé pour construire des arbres de décision
- appartient à la famille TDIDT (*Top Down Induction of Decision Trees*)
- basé sur le gain d'information et le critère d'entropie :

$$I = -\sum_{k=1}^{K} p_k \times \log p_k$$

Gain d'information et critère d'entropie (1/2)

Mesurer l'information

- théorie de l'information de Shannon (1949)
- exprime la quantité d'information (nombre de *bits*) pour coder l'information

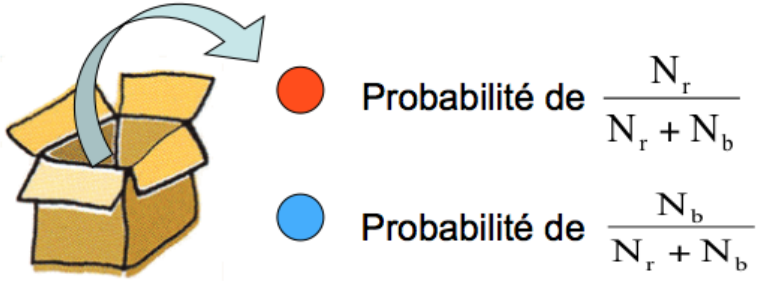
Gain d'information et critère d'entropie (1/2)

Mesurer l'information

- théorie de l'information de Shannon (1949)
- exprime la quantité d'information (nombre de *bits*) pour coder l'information

Mesurer l'information

Prenons par exemple une urne avec des boules de deux couleurs :



Gain d'information et critère d'entropie (2/2)

Coder l'information

- P bits permettent de coder 2^P informations
- $log(P)$ bits permettent de coder P informations
- connaître la couleur de la boule apporte :

$$log_2(N) - log_2(N_r) \text{ bits}$$

- l'information acquise est donc de :

$$p(rouge)[log_2(N) - log_2(N_r)] + p(bleu)[log_2(N) - log_2(N_b)]$$

Gain d'information et critère d'entropie (2/2)

Coder l'information

- P bits permettent de coder 2^P informations
- $log(P)$ bits permettent de coder P informations
- connaître la couleur de la boule apporte :

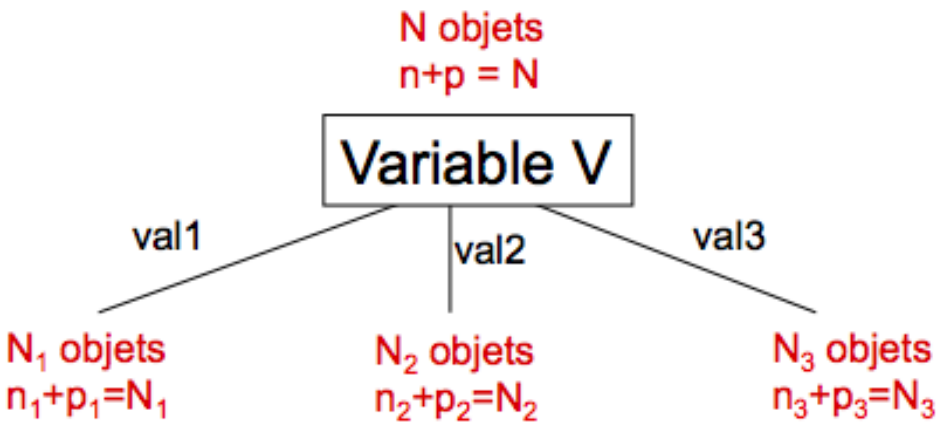
$$log_2(N) - log_2(N_r) \text{ bits}$$

- l'information acquise est donc de :

$$p(rouge)[log_2(N) - log_2(N_r)] + p(bleu)[log_2(N) - log_2(N_b)]$$

Le critère à maximiser dans l'algorithme ID3 est alors $I - E(N, A)$, où $E(N, A) = \frac{N_1}{N} I_1 + \frac{N_2}{N} I_2 + \frac{N_3}{N} I_3$

Illustration



$$Gain = I - E(N, A) = I - (\frac{N_1}{N} I_1 + \frac{N_2}{N} I_2 + \frac{N_3}{N} I_3)$$

Arbre binaire de classification

Principe

- arbre binaire de classification : succession de noeud
- noeud : défini par le choix d'une variable et d'une division $\Rightarrow$ partition en 2 classes
- division : choix d'un seuil (variable explicative quanti.) ou d'un groupe de modalités (variable explicative quali.)
- racine (noeud initial) : ensemble de l'échantillon ; la procédure est itérée pour chacun des 2 fils du noeud init. Et ainsi de suite.

Paramétrisation de l'algorithme

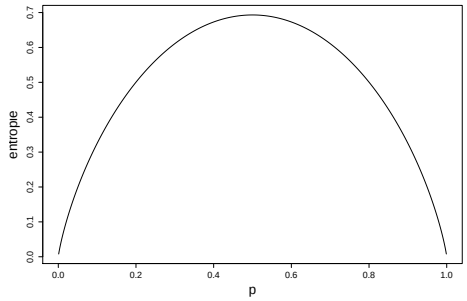
- critère de qualité d'une division
- règle permettant de dire si un noeud est terminal (feuille)
- affectation de chaque feuille à l'une des K classes

Arbre binaire de classification

Critère de division

- D : critère d'hétérogénéité d'un noeud
  - critère d'entropie

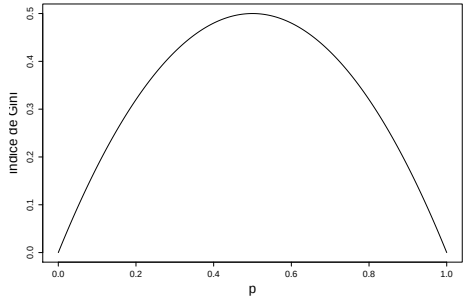
$$D = -2n \sum_{k=1}^{K} \frac{n_k}{n} \log \frac{n_k}{n}$$



où n est le nombre d'éléments du noeud, n_k celui de la classe k, et avec la convention $0 \log 0 = 0$

  - indice de Gini (une comparaison est discutée ici)

$$D = -\sum_{k=1}^{K} \frac{n_k}{n} \left(1 - \frac{n_k}{n}\right)$$



- on retient la division qui maximise

$$D_{\text{noeud}} - (D_{\text{fils gauche}} + D_{\text{fils droit}})$$

Arbre binaire de classification

Règle définissant un noeud terminal

- si un des noeuds fils est vide
- si le nombre d'observation est inférieur à un seuil (entre 1 et 5)

Règle d'affectation d'un noeud à une classe

- la classe majoritaire
- la classe a posteriori la plus probable (si des probabilités a priori sont connues)
- la classe la moins coûteuse (si des coûts de mauvais classements sont connus)

Arbre binaire de classification

Construction d'un arbre optimal

- les arbres ainsi créés sont souvent très raffinés et donc instable (sur-ajustement)
- on va recherche des arbres plus parcimonieux : méthode CART (Breinman et al. 1984)

Algorithm 1 CART

1: construction de l'arbre maximal
2: construction d'une séquence d'arbre emboités
   - on compare tous les arbres obtenus en supprimant une des dernières divisions (et donc une feuille) suivant un critère d'erreur de classement, et on retient le meilleur
   - ainsi de suite en remontant jusqu'à la racine
3: comparer les arbres obtenus via un taux de mauvais classement estimé sur échantillon test ou par validation croisée

Arbre binaire de classification

Avantages

- règle d'affectation lisible
- aucune hypothèse paramétrique
- robuste vis-à-vis des données atypiques
- aller au-delà de la classification

Inconvénients

- demande des tailles d'échantillons grandes
- coûteux en temps de calcul
- parfois instable

Méthode d'ensemble : utiliser une forêt d'arbres

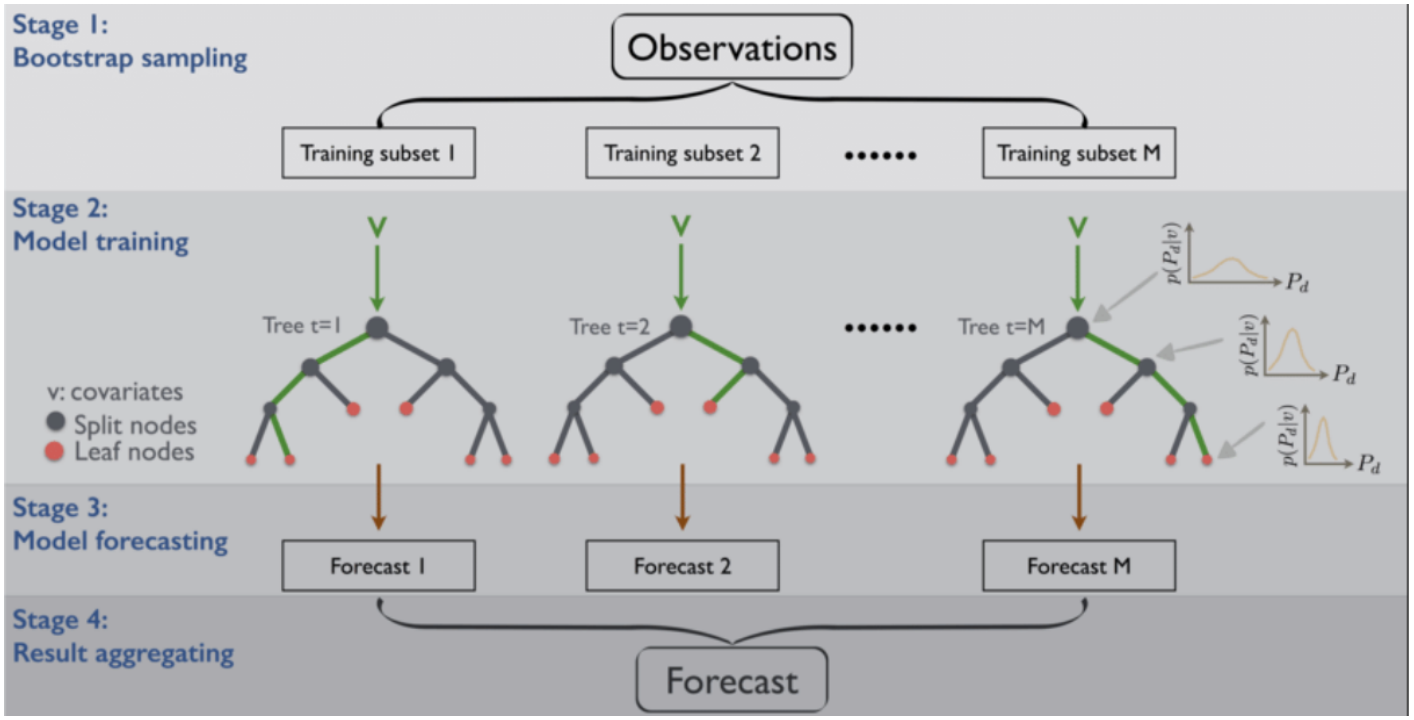


Illustration de :
He, Xiaogang, et al. "Spatial downscaling of precipitation using adaptable random forests." Water resources research 52.10 (2016): 8217-8237.

Méthode d'ensemble : utiliser une forêt d'arbres

- Principe du Bagging = Bootstrap sampling + Aggrégation
- La décision est prise par un vote à la majorité d'un ensemble de classifieurs (ici, des arbres).
- L'un des algorithmes historiques est Adaboost (Freund & Shapire, 1995)
- L'algorithme le plus connu et le plus employé aujourd'hui est XGBoost (Tianqi & Guestrin, 2016)

Freund Y. and Schapire R.E. (1995), A decision-theoretic generalization of on-line learning and an application to boosting. LNCS, Springer.
Tianqi C. and Guestrin C. (2016), Xgboost: A scalable tree boosting system. SIGKDD International Conference on Knowledge Discovery and Data mining

XGBoost (ici, pour la régression)

- La prédiction est une combinaison linéaire d'un ensemble d'arbres de décision $\mathcal{F}$:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), f_k \in \mathcal{F}$$

- La fonction à optimiser est la suivante :

$$L = \sum_{i}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \omega(f_k)$$

où ω est une fonction qui mesure la complexité des arbres.

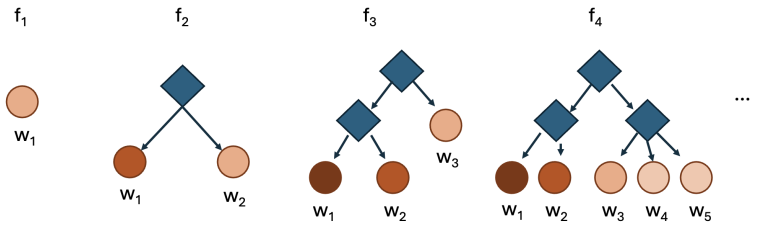
- Adoptant une approche additive et l'erreur MSE, la fonction objectif devient:

$$L_t = \sum_{i}^{n} [g_i.f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] \omega(f_t)$$

où g_i est la dérivée première de l et h_i la dérivée seconde.

XGBoost (suite)

- La construction de $\mathcal{F}$ pour optimiser L se fait de manière heuristique, un arbre à la fois :



- Pour aller plus loin, il faut mieux définir f_t et ω :

$$f_t(x) = w_{q(x)}, w \in \mathbb{R}^T, q : \mathbb{R}^d \to \{1, 2 ... T\}$$

$$\omega(f) = \gamma . T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

où T est le nombre de feuilles, γ et λ des hyperparamètres à fixer.

XGBoost (suite)

- On arrive à une formulation qui permet de combiner les prédictions du modèle (aux feuilles) et la complexité de la structure :

$$L_o = \sum_{j=1}^{T} [G_j.w_j + \frac{1}{2}(H_j + \lambda)w_j^2] + \gamma T$$

où $G_j = \sum_{i \in I_j} g_i$ et $H_j = \sum_{i \in I_j} h_i$, I_j étant l'ensemble des objets traités par la feuille $j, j \in \{1, 2 ... T\}$.

- On optimise selon w avec la formule suivante :

$$\Delta_L = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T$$

- Plus de détails :
`https://xgboost.readthedocs.io/en/stable/tutorials/model.html`

XGBoost : pour conclure

Avantages

- Très performant et rapide en inférence, en particulier pour les données tabulaires
- Robuste aux données manquantes et aberrantes
- Inclus un terme de régularisation pour éviter le sur-apprentissage
- Passe facilement à l'échelle
- Possibilités d'interpréter les prédictions

Désavantages

- Nécessite un réglage fin des (hyper)paramètres
- L'entraînement peut être long, surtout avec des gros jeux de données
- Pas très adapté aux espaces en très grande dimension (surtout s'ils sont creux)

Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

Régression logistique (RL)

On se place dans le cas $K = 2$ (régression logistique dichotomique)

$$Y \in \{0, 1\}$$

Les variables explicatives peut-être quantitatives ou qualitatives :

- $X_j \in \mathbb{R}$: utilisée telle quelle
- $X_j \in \{m_j^1, \ldots, m_j^{k_j}\}$: transformée en variables indicatrices

$$X_j \Rightarrow \left(\mathbb{1}_{X_j = m_j^1}, \ldots, \mathbb{1}_{X_j = m_j^{k_j - 1}} \right)$$

RL : le modèle

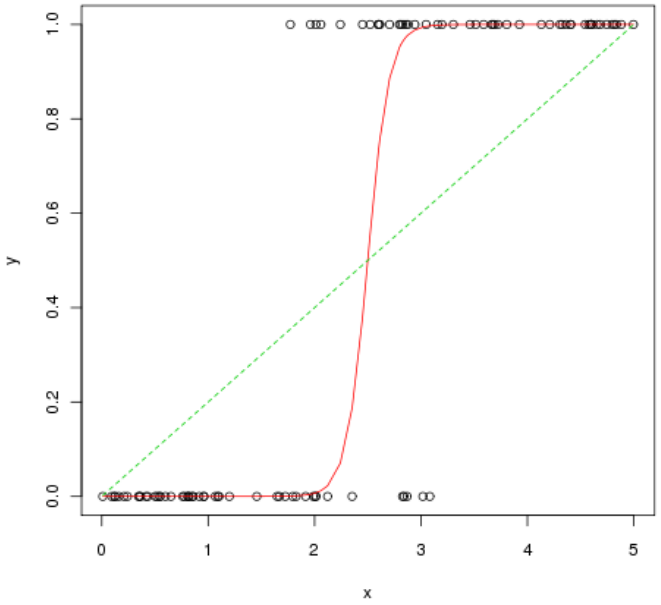
Principe

- on cherche à modéliser de façon linéaire $\ln \frac{P(Y=1|\mathbf{X}=\mathbf{x})}{1-P(Y=1|\mathbf{X}=\mathbf{x})}$:

$$\ln \frac{P(Y=1|\mathbf{X}=\mathbf{x})}{1-P(Y=1|\mathbf{X}=\mathbf{x})} = \underbrace{\ln \frac{\pi(\mathbf{x})}{1-\pi(\mathbf{x})}}_{logit(\pi(\mathbf{x}))} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

ce modèle s'écrit aussi

$$\pi(\mathbf{x}) = \frac{\exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right)}{1 + \exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right)}$$



RL : Interprétation via les Odds Ratio

On définit deux outils d'interprétation des paramètres du modèle :

- **odds** : combien de fois on a plus de chance d'avoir $Y = 1$ au lieu d'avoir $Y = 0$ lorsque $\mathbf{X} = \mathbf{x}$:

$$\text{odds}(\mathbf{x}) = \frac{\pi(\mathbf{x})}{1 - \pi(\mathbf{x})}$$

- **odds-ratio** : combien de fois on a plus de chance d'avoir $Y = 1$ au lieu d'avoir $Y = 0$ lorsque $\mathbf{X} = \mathbf{x}_i$ au lieu de $\mathbf{X} = \mathbf{x}_j$

$$\text{odds-ratio}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\text{odds}(\mathbf{x}_i)}{\text{odds}(\mathbf{x}_j)}$$

RL : Exemple d'interprétation

- Y : présence / absence d'un cancer des poumons,
- X : fumeur / non fumeur.

Exemple de données (fictives)

- Chez les fumeurs : $P(Y = 1|X = \text{fumeur}) = 0.01$ et $P(Y = 0|X = \text{fumeur}) = 0.99$.
- $\Rightarrow$ odds$(X = \text{fumeur}) = 1/99$: on a une *chance* sur 99 d'avoir un cancer des poumons lorsque l'on est fumeur.
- Chez les non fumeurs : $P(Y = 1|X = \text{non fumeur}) = 10^{-4}$
- $\Rightarrow$ odds-ratio$(\text{fumeur}, \text{non fumeur}) = \frac{1/99}{1/9999} = 101$: on a 101 fois plus de chance d'avoir un cancer des poumons pour un fumeur que pour un non fumeur.

RL : Prédictions

Pour une nouvelle observation $\mathbf{x}^*$, le modèle nous prédit le **score**

$$P(Y = 1|\mathbf{X} = \mathbf{x}^*; \hat{\beta}) = \frac{\exp\left(\hat{\beta}_0 + \sum_{j=1}^{p} \hat{\beta}_j x_j\right)}{1 + \exp\left(\hat{\beta}_0 + \sum_{j=1}^{p} \hat{\beta}_j x_j\right)}$$

On peut alors définir la règle de classement suivante :

$$\hat{y}_s^* = \begin{cases} 1 & \text{si } P(Y = 1|\mathbf{X} = \mathbf{x}^*; \hat{\beta}) \geq s \\ 0 & \text{sinon} \end{cases}$$

où s est un seuil défini par en collaboration avec le praticien.

Si $s = 0.5$, on obtient la règle du **maximum a posteriori** (MAP).

Rq : souvent le score est plus intéressant que la prédiction elle même (credit scoring, médecine...)

Rég. logistique polytomique ou ordinale ($K > 2$)

- régression logistique **polytomique** : $Y \in \{1, \ldots, K\}$
  - on réalise $K-1$ régressions logistiques de $\pi_k(\mathbf{x})$ versus $\pi_K(\mathbf{x})$:

$$\ln \frac{\pi_k(\mathbf{x})}{\pi_K(\mathbf{x})} = \beta_{0k} + \sum_{j=1}^{p} \beta_{jk} x_j \qquad \forall 1 \leq k \leq K-1.$$

  où $\pi_k(\mathbf{x}) = P(Y = k | \mathbf{X} = \mathbf{x})$.

- régression logistique **ordinale** : $Y \in \{1, \ldots, K\}$
  $\Rightarrow$ *les modalités 1,. . . ,K de Y sont ordonnées*
  - on modélise généralement des *logits cumulatifs* :

$$\ln \frac{\pi_{k+1}(\mathbf{x}) + \ldots + \pi_K(\mathbf{x})}{\pi_1(\mathbf{x}) + \ldots + \pi_k(\mathbf{x})} = \beta_{0k} + \sum_{j=1}^{p} \beta_{jk} x_j \qquad \forall 1 \leq k \leq K-1.$$

Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

Support Vector Machines

- Techniques issues de l'apprentissage statistique (Cortes and Vapnik,1995)
- Classifieur basé sur la découverte de vecteurs supports
- Idée de marge, d'où une traduction possible des SVM : « Séparateurs à Vastes Marges »
- Adapté à l'utilisation de noyaux pour la classification non linéaire
- Très bons résultats dans la pratique

Why SVM Is Still Useful in 2025?

Despite the rise of deep learning, SVM excels in specific areas:

- Works well with small to medium-sized datasets – Unlike deep learning, SVM doesn't require millions of examples.
- Great for structured/tabular data – Ideal for finance, healthcare, cybersecurity, and more.
- Interpretable & Explainable – Unlike neural networks, SVM decisions can be easily understood.
- Less computationally expensive – Efficient when deep learning is overkill.

https://medium.com/@thevisionaryvectorsblog/svm-in-2025-old-school-or-still-cool-baf7a3db95ef

Why SVM Is Still Useful in 2025?

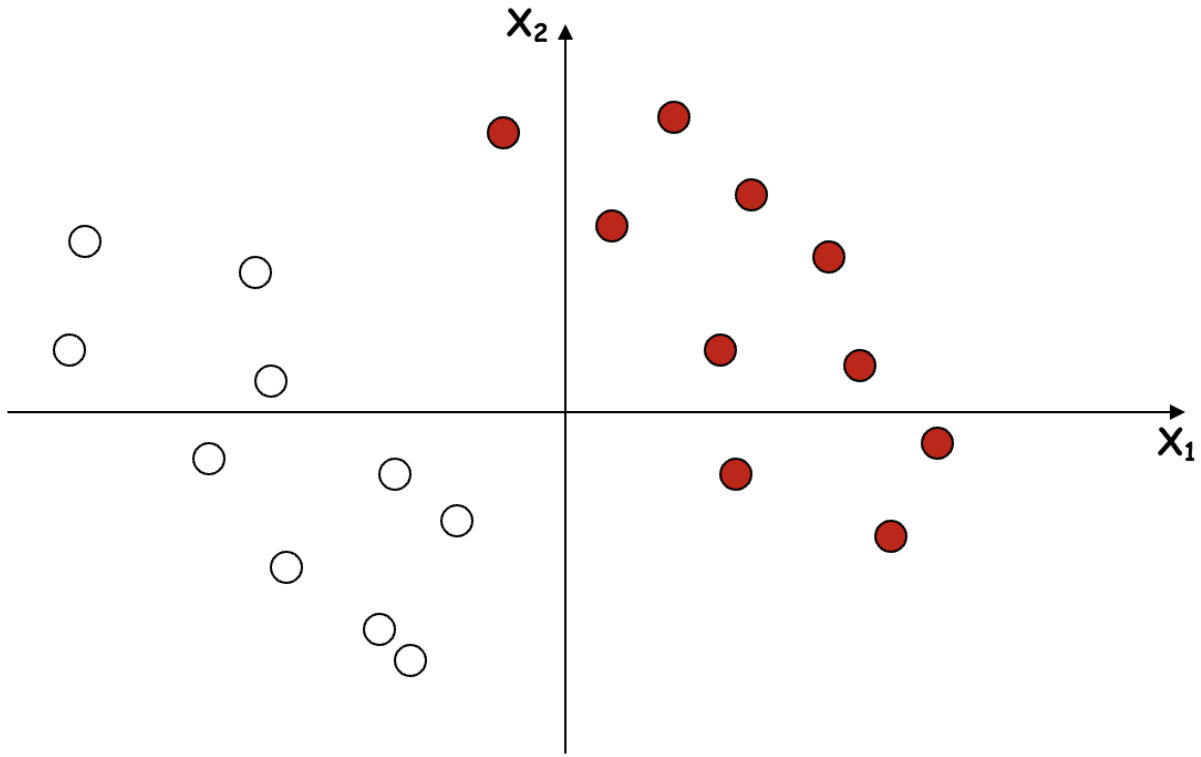
Industry	Use Case	Why SVM?
Healthcare	Disease detection (e.g., cancer)	Works well with structured medical imaging data
Finance	Credit scoring, fraud detection	Handles structured financial records efficiently
Cybersecurity	Intrusion detection, malware classification	Quickly classifies network anomalies
Manufacturing	Defect detection, predictive maintenance	Effective for sensor data and quality control
Agriculture	Crop disease detection	Performs well on image-based classification
Astronomy	Star classification, exoplanet discovery	Works with high-dimensional space data

Why Generative AI Can’t Replace SVM in These Use Cases

Use Case	Why GenAI Falls Short	Why SVM Works
Fraud Detection	GenAI requires vast labeled data and can hallucinate results	SVM works with structured transaction data
Medical Diagnosis	GenAI struggles with regulatory compliance & interpretability	SVM provides clear, explainable decisions
Cybersecurity	Deep learning models are prone to adversarial attacks	SVM offers robust and reliable classification

SVM dans le cas linéaire : illustration

Les deux classes sont linéairement séparables :

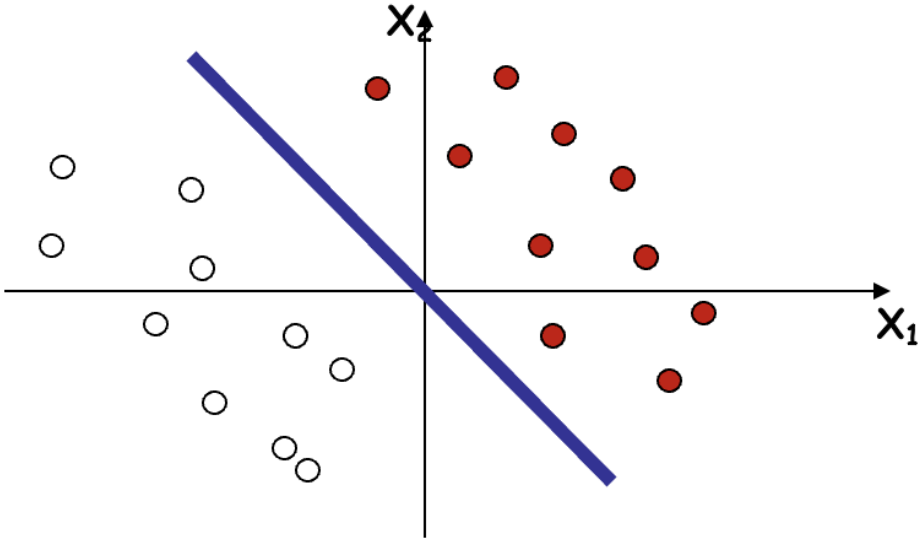


Principe des SVM (1)

Soient les N données d'entraînement :

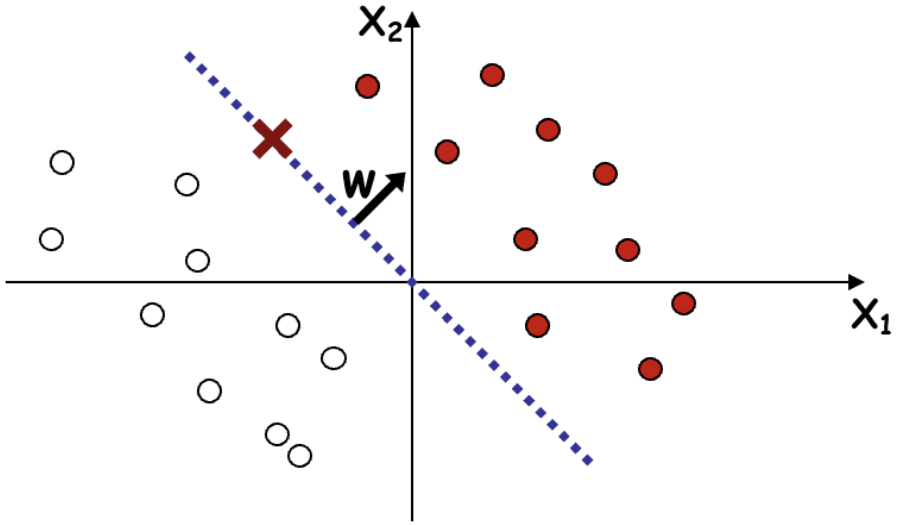
- $\mathbf{X} = \{x_i = (x_i^1, \ldots, x_i^p) \in \mathbb{R}^p\}$
- $\mathbf{Y} = (y_1, \ldots, y_N)$ est le numéro de la classe de l'individu i
- On a donc un ensemble de tuples $\{(x_i, y_i), i = 1 \ldots N, y_i \in -1, 1, x_i \in \mathbb{R}^p\}$

On suppose qu'il existe un hyperplan qui sépare les exemples positifs $\mathbf{X}+$ des exemples négatifs $\mathbf{X}-$.

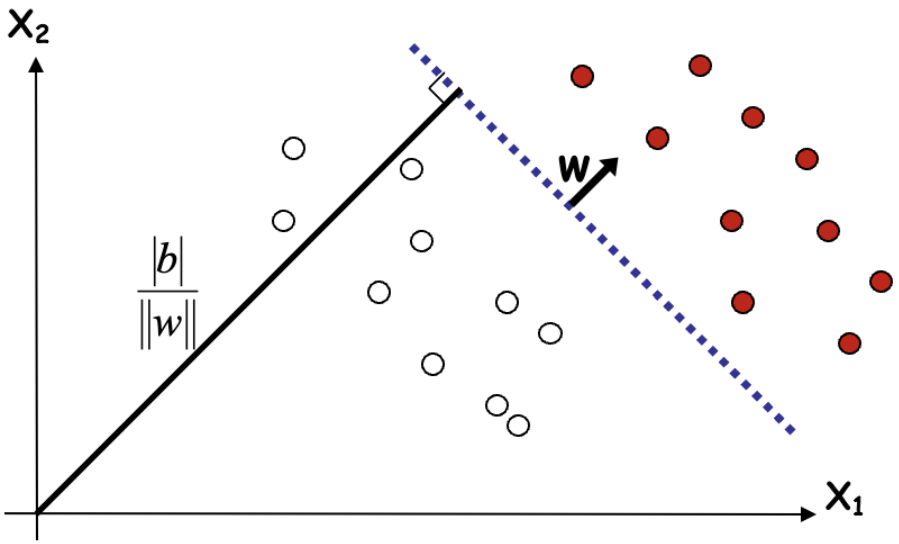


Principe des SVM (2)

- Les points qui se trouvent sur cet hyperplan vérifient : $\boldsymbol{w}.\mathbf{x} + \mathbf{b} = 0$
- Le vecteur $\boldsymbol{w}$ est normal à l'hyperplan.



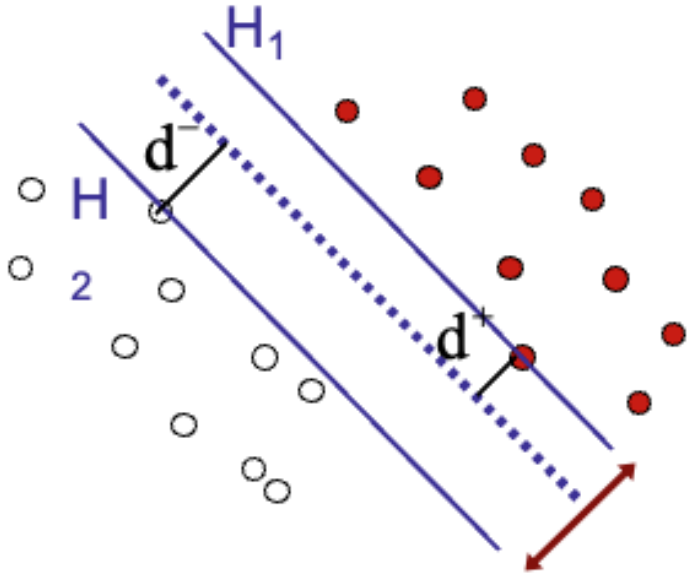
Principe des SVM (3)



- La distance de l'hyperplan à l'origine est $\frac{|\mathbf{b}|}{||\boldsymbol{w}||}$, où $||\boldsymbol{w}||$ est la norme du vecteur $\boldsymbol{w}$.
- Considérons à présent les exemples (positifs et négatifs) les plus proches de cet hyperplan.

Principe des SVM (4)

- Soient d^+ et d^- les distances séparant (resp.) l'exemple positif (négatif) le plus proche de l'hyperplan.
- La marge de l'hyperplan séparateur est $d^+ + d^-$.



Objectif d'un SVM

- Trouver l'hyperplan avec la plus grande (vaste) marge pour séparer les données.
- En utilisant la forme canonique, l'hyperplan doit satisfaire les contraintes :

$$x_i.w + b \geq +1 \text{ pour } y_i = +1$$

$$x_i.w + b \leq -1 \text{ pour } y_i = -1$$

- Cela peut se réécrire :

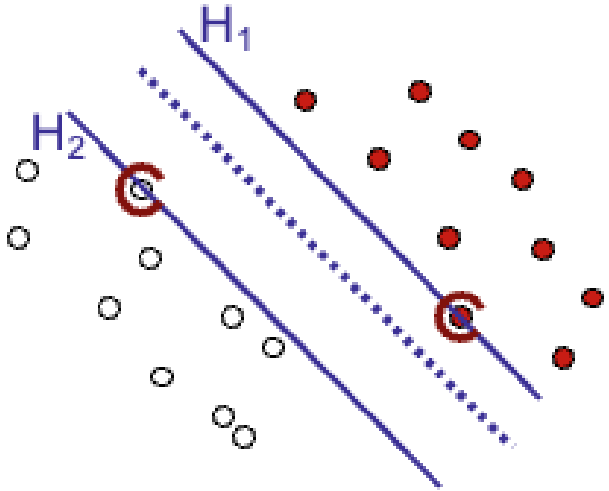
$$\forall_i \; y_i(x_i.w + b) - 1 \geq 0$$

Problème d'optimisation

- Les points sur H_1 vérifient $x_i.w + b = 1$.
- Les points sur H_2 vérifient $x_i.w + b = -1$.
- Ainsi, $d^+ = d^- = \frac{1}{||w||}$ et la marge est de $\frac{2}{||w||}$
- Le problème revient à minimiser $||w||^2$ sous les contraintes : $\forall_i \; y_i(x_i.w + b) - 1 \geq 0$
- Il s'agit d'un problème d'optimisation avec contraintes linéaires.
- La fonction est convexe : il existe une seule solution globale qu'on peut trouver efficacement.

Et les vecteurs supports ?

- Il s'agit des exemples situés sur l'un ou l'autre des hyperplans H^1 et H^2 :



- Ils sont fondamentaux car, si on les retire, la solution trouvée en est naturellement modifiée.

Résolution de la forme primaire

Résolution de la forme primaire

- Il faut régler $(d+1)$ paramètres, d étant la dimension de l'espace des représentations.
- C'est possible lorsque d est assez petit avec des méthodes d'optimisation quadratique classiques.
- Cela devient impossible lorsque d est trop grand.

Formulation lagrangienne

- Les contraintes vont être remplacées par des contraintes sur les coefficients lagrangiens.
- Les données d'entraînement vont uniquement être utilisées sous la forme d'un produit scalaire entre vecteurs de la forme $< x_i.x_j >$.
- Cette dernière propriété est très importante car elle va nous permettre de généraliser le procédé au cas non linéairement séparables !

Conditions de Kuhn-Tucker

- Introduction des multiplicateurs lagrangiens $\alpha_i \geq 0, i = 1..n$ pour chaque contrainte $\forall_i \; y_i(x_i.w + b) - 1 \geq 0$
- Cela donne la fonction à minimiser :

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

- L_P doit être minimisée sous les contraintes :
  - sa dérivée par rapport aux α_i s'annule,
  - $\forall_i, \; i = 1..n, \alpha_i \geq 0$

Formulation duale (1)

- Il s'agit d'un problème de programmation quadratique convexe qui peut se transformer en un problème dual via la forme de Wolfe : *maximiser* L_P sous les contraintes :
  - le gradient de LP (resp. à w et b) s'annule,
  - $\forall_i,\ i = 1..n, \alpha_i \geq 0$
- **Propriété fondamentale :** le maximum de L_P avec ce deuxième ensemble de contraintes survient *aux mêmes valeurs* de w, b et α que le minimum de L_P avec le premier ensemble de contraintes.

Formulation duale (2)

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

$\nabla L_p = 0$ implique :

Formulation duale (2)

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

$\nabla L_p = 0$ implique :

$\frac{\partial L_p}{\partial b} = \sum_{i=1}^{n} \alpha_i y_i$ $\qquad$ $\sum_{i=1}^{n} \alpha_i y_i = 0$

Formulation duale (2)

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

$\nabla L_p = 0$ implique :

$\frac{\partial L_p}{\partial b} = \sum_{i=1}^{n} \alpha_i y_i \qquad \sum_{i=1}^{n} \alpha_i y_i = 0$

$\frac{\partial L_p}{\partial w} = w - \sum_{i=1}^{n} \alpha_i y_i x_i \qquad w = \sum_{i=1}^{n} \alpha_i y_i x_i$

Formulation duale (3)

On réinjecte $\sum_{i=1}^{n} \alpha_i y_i = 0$ et $w = \sum_{i=1}^{n} \alpha_i y_i x_i$ dans L_P :

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

Formulation duale (3)

On réinjecte $\sum_{i=1}^{n} \alpha_i y_i = 0$ et $w = \sum_{i=1}^{n} \alpha_i y_i x_i$ dans L_P :

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

Cela donne :

$$L_D = \sum_{i=1}^{n} \alpha_i + \frac{1}{2}(\sum_{i=1}^{n} \alpha_i y_i x_i)^2 - \sum_{i=1}^{n} \alpha_i y_i x_i.(\alpha_i y_i x_i) - b \sum_{i=1}^{n} \alpha_i y_i$$

Formulation duale (3)

On réinjecte $\sum_{i=1}^{n} \alpha_i y_i = 0$ et $w = \sum_{i=1}^{n} \alpha_i y_i x_i$ dans L_P :

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

Cela donne :

$$L_D = \sum_{i=1}^{n} \alpha_i + \frac{1}{2}(\sum_{i=1}^{n} \alpha_i y_i x_i)^2 - \sum_{i=1}^{n} \alpha_i y_i x_i.(\alpha_i y_i x_i) - b \sum_{i=1}^{n} \alpha_i y_i$$

$$L_D = \sum_{i=1}^{n} \alpha_i + \frac{1}{2}(\sum_{i=1}^{n} \alpha_i y_i x_i)^2 - (\sum_{i=1}^{n} \alpha_i y_i x_i)^2$$

Formulation duale (3)

On réinjecte $\sum_{i=1}^{n} \alpha_i y_i = 0$ et $w = \sum_{i=1}^{n} \alpha_i y_i x_i$ dans L_P :

$$L_P = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i y_i (x_i.w + b) + \sum_{i=1}^{n} \alpha_i$$

Cela donne :

$$L_D = \sum_{i=1}^{n} \alpha_i + \frac{1}{2}(\sum_{i=1}^{n} \alpha_i y_i x_i)^2 - \sum_{i=1}^{n} \alpha_i y_i x_i.(\alpha_i y_i x_i) - b \sum_{i=1}^{n} \alpha_i y_i$$

$$L_D = \sum_{i=1}^{n} \alpha_i + \frac{1}{2}(\sum_{i=1}^{n} \alpha_i y_i x_i)^2 - (\sum_{i=1}^{n} \alpha_i y_i x_i)^2$$

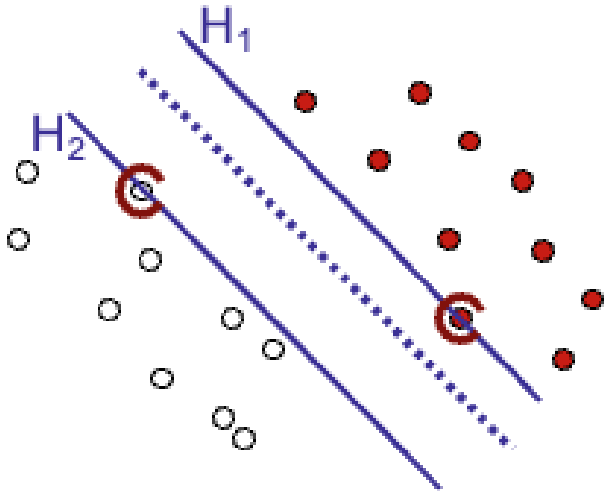
$$L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}(\sum_{i=1}^{n} \alpha_i y_i x_i)^2 = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j (x_i.x_j)$$

Nouveau problème d'optimisation

- Maximiser : $L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j (x_i . x_j)$
  sous les contraintes :
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $\forall_i,\ i = 1..n, \alpha_i \geq 0$
- L'apprentissage d'un SVM, dans le cas linéaire, revient donc à maximiser L_D relativement aux multiplicateurs α_i.
- Remarque : il y a bien un coefficient α_i *par exemple* dans la base d'apprentissage.

Vecteurs supports

- Les points associés à un coefficient $\alpha_i > 0$ sont appelés les **vecteurs supports**.



- Tous les autres points ont un coefficient $\alpha_i = 0$.
- Les vecteurs supports sont très importants pour l'apprentissage car ils sont très proches de la frontière de décision.

Résolution du problème d'optimisation

- Pour les petits jeux de données, il suffit de recourir aux méthodes classiques d'optimisation de fonctions quadratiques convexes sous contraintes linéaires (cf. par exemple Jorge J. Moré and Stephen J. Wright. Optimization Software Guide. SIAM, 1993).
- Pour de plus grands jeux de données, il faut faire appel à d'autres techniques de programmation non linéaire sous contraintes (cf. par exemple R. Fletcher. Pratical Methods of Optimization. John Wiley and Sons, Inc., 2nd edition, 1987)

Classification avec un SVM

- Une fois qu'un SVM a été entraîné, il suffit de calculer de "quel côté de la frontière" se trouve un nouvel exemple x_{new}.
- Pour cela, on utilise bien sûr l'hyperplan que l'on vient de trouver :

$$h(x_{new}) = sgn(w.x_{new} + b)$$

avec $w = \sum_{i=1}^{n} \alpha_i y_i x_i \qquad \Longleftrightarrow \qquad w = \sum_{i=1}^{N_S} \alpha_i y_i x_i$

où N_S désigne l'ensemble des vecteurs supports.

Classification avec un SVM

- Une fois qu'un SVM a été entraîné, il suffit de calculer de "quel côté de la frontière" se trouve un nouvel exemple x_{new}.
- Pour cela, on utilise bien sûr l'hyperplan que l'on vient de trouver :

$$h(x_{new}) = sgn(w.x_{new} + b)$$

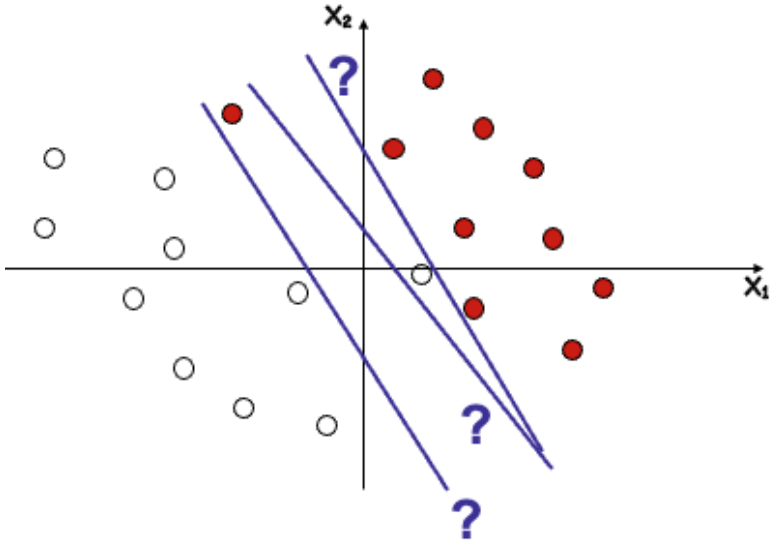
avec $w = \sum_{i=1}^{n} \alpha_i y_i x_i \qquad \Longleftrightarrow \qquad w = \sum_{i=1}^{N_S} \alpha_i y_i x_i$

où N_S désigne l'ensemble des vecteurs supports.

et b est calculé à l'aide de l'équation (cf. KKT) :
$\forall_i, i = 1..n, \alpha_i(y_i(w.x_i + b) - 1) = 0$

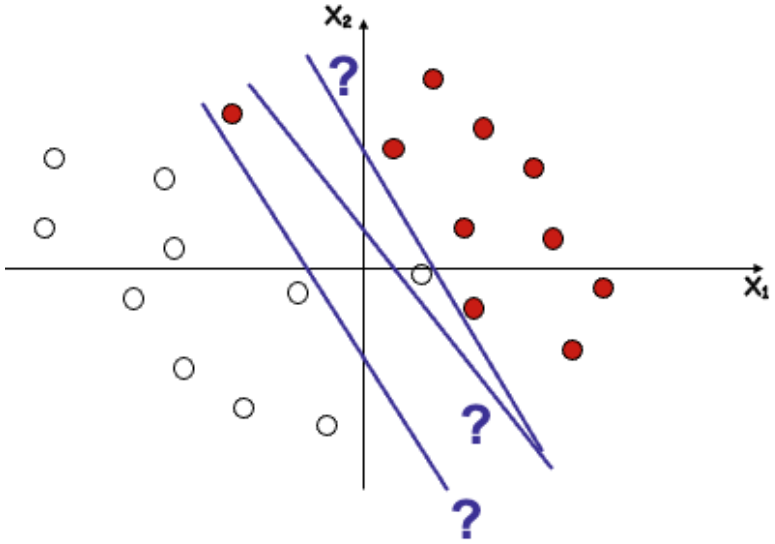
Le cas de données non linéairement séparables

Imaginons le cas suivant :



Le cas de données non linéairement séparables

Imaginons le cas suivant :



Il n'existe pas de solution à notre problème d'optimisation...

Relâcher les contraintes : les marges “molles”

- Reprenons les contraintes précédentes :
  - $x_i.w + b \geq +1$ pour $y_i = +1$
  - $x_i.w + b \leq -1$ pour $y_i = -1$
- Nous introduisons un **coût supplémentaires** à l’aide de variable de relaxation xi_i :
  - $x_i.w + b \geq +1 - \xi_i$ pour $y_i = +1$
  - $x_i.w + b \leq -1 + \xi_i$ pour $y_i = -1$

  avec la contrainte :
  $\forall_i, i = 1..n, \xi_i \geq 0$

Relâcher les contraintes (2)

- Une *erreur* survient donc lorsque, pour un exemple x_i, sa valeur correspondante ξ_i dépasse 1.
- Le nombre d'erreurs réalisées est donc borné par : $\sum_i \xi_i$
- La fonction à minimiser est donc modifiée :

$$\frac{1}{2}||w||^2 + C(\sum_i \xi_i)^k$$

où k et C sont deux constantes à fixer.

Cas avec $k = 1$

La forme duale de Wolfe, qu'il faut maximiser, devient alors :

$$L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j (x_i . x_j)$$

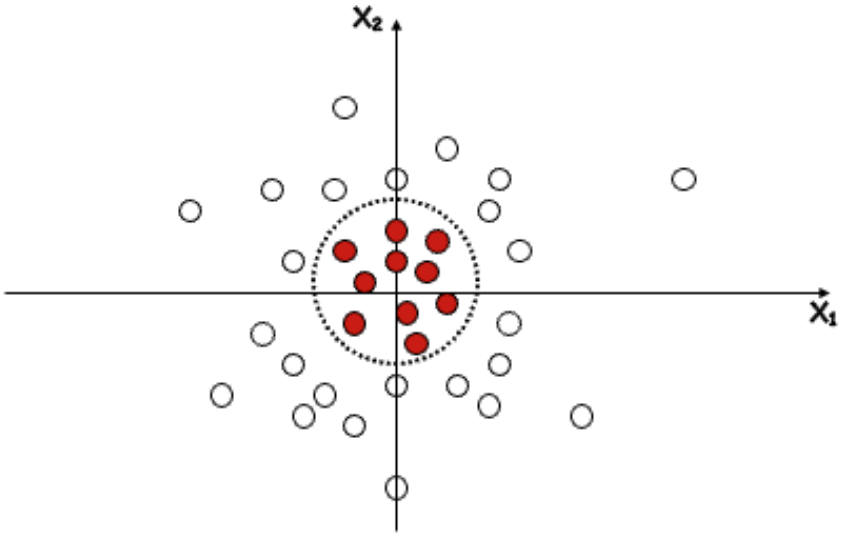
sous les contraintes :

- $\sum_{i=1}^{n} \alpha_i y_i = 0$
- $\forall_i,\ i = 1..n, 0 \leq \alpha_i \leq C$

et la solution est toujours données par : $w = \sum_{i=1}^{n} \alpha_i y_i x_i$

Données non linéairement séparables (suite)

Imaginons à présent le cas suivant :

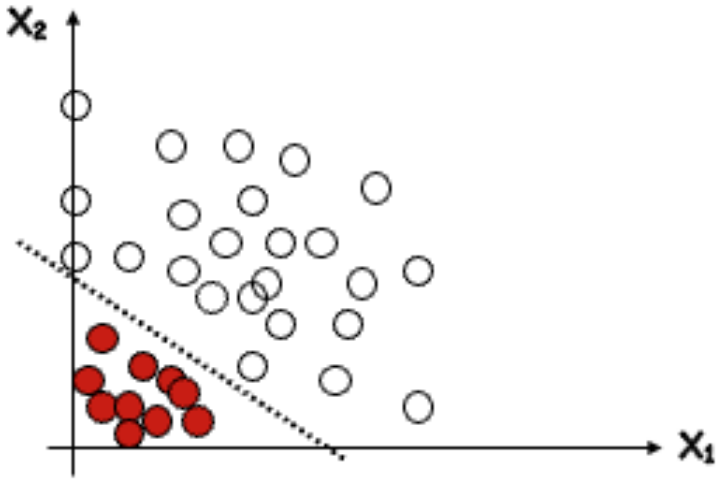


Si on utilise un nouveau espace de représentations des données

- Supposons l'application suivante :

$$\chi(x_i, x_j) = (x_i^2, x_j^2)$$

- Alors notre problème revient à séparer les données dans ce nouvel espace de représentation :



Remarque préliminaire

- Rappel de l'équation du lagrangien :

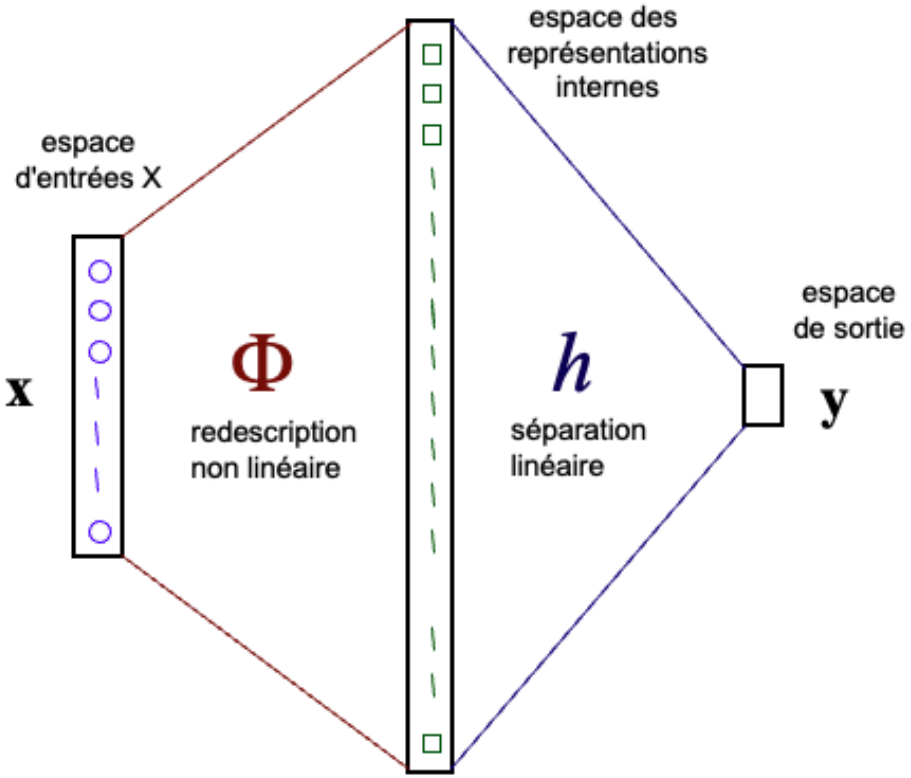
$$L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j (x_i . x_j)$$

- On remarque l'utilisation du produit scalaire $< x_i . x_j >$. En fait, on n'utilise les vecteurs exemples que sous cette forme.
- Cette remarque est primordiale pour pouvoir utiliser le "truc" qui est à la base des SVM non linéaires (Aizerman,1964).

SVM non linéaires

- **Idée de base :** si l'on projette les données dans un espace de description suffisamment grand, alors le problème devient linéairement séparable.
- Mais cela pose de nouveaux problèmes, dont celui de la malédiction de la dimension.
- Pour résoudre tout de même notre problème, nous allons utiliser le "truc" du **noyau** basé sur le fait que notre fonction à optimiser calcule le produit scalaire $< x_i.x_j >$.

Redescription des données



Redescription dans $\mathcal{H}$

- On suppose une application Φ qui re-décrit les exemples de X (par exemple dans $\mathbb{R}^d$) dans un nouvel espace $\mathcal{H}$:

$$\Phi : \mathbb{R}^d \rightarrow \mathcal{H}$$

Dans ce nouvel espace, la forme duale du lagrangien se réécrit :

$$L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j (\Phi(x_i).\Phi(x_j))$$

avec les contraintes :

- $\sum_{i=1}^{n} \alpha_i y_i = 0$
- $\forall_i,\ i = 1..n, 0 \leq \alpha_i \leq C$

Fonction noyau K

- Notre nouveau problème d'optimisation $\arg\max(L_D)$ se résume à savoir calculer $< \Phi(x_i).\Phi(x_j) >$.
- Calculer le produit scalaire est immédiat *à condition* que l'on connaisse la fonction Φ.
- Si l'on ne connaît pas Φ, le calcul n'est pas impossible mais plus difficile. La solution est de postuler l'existence d'une fonction noyau K telle que : $K(x_i, x_j) = \Phi(x_i).\Phi(x_j)$.

Problème d'optimisation dans le cas non linéaire

Maximiser :

$$L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

sous les contraintes :

- $\sum_{i=1}^{n} \alpha_i y_i = 0$
- $\forall_i,\ i = 1..n, 0 \leq \alpha_i \leq C$

Problème d'optimisation dans le cas non linéaire

Maximiser :

$$L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

sous les contraintes :

- $\sum_{i=1}^{n} \alpha_i y_i = 0$
- $\forall_i,\ i = 1..n, 0 \leq \alpha_i \leq C$

Le SVM construit sépare linéairement les données dans l'espace $\mathcal{H}$, mais non linéairement dans l'espace initial X, alors même que l'on ne connaît pas la fonction Φ !

Problème d'optimisation dans le cas non linéaire

Maximiser :

$$L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1,j}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

sous les contraintes :

- $\sum_{i=1}^{n} \alpha_i y_i = 0$
- $\forall_i,\ i = 1..n, 0 \leq \alpha_i \leq C$

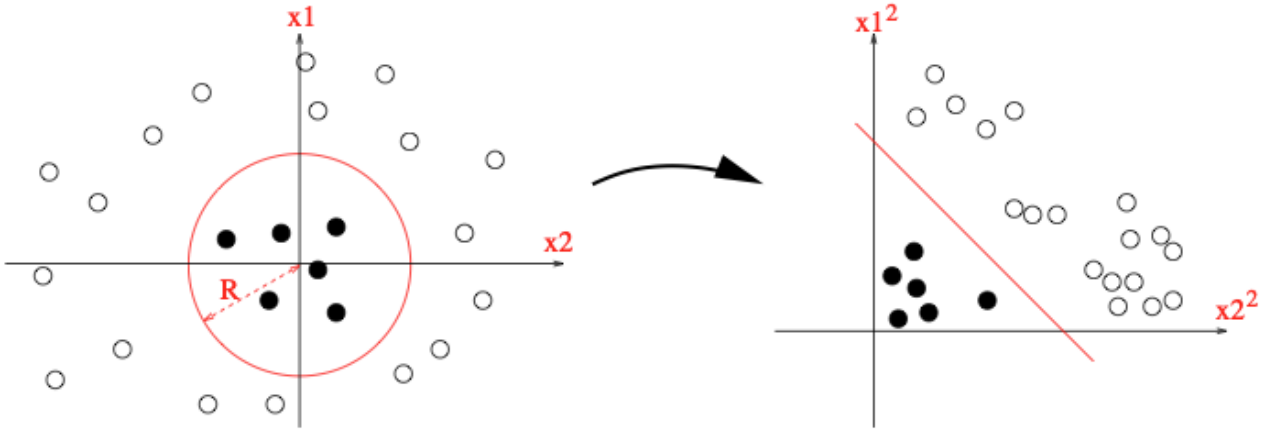
Le SVM construit sépare linéairement les données dans l'espace $\mathcal{H}$, mais non linéairement dans l'espace initial X, alors même que l'on ne connaît pas la fonction Φ !

La décision se base toujours sur :

$$h(x_{new}) = sgn(\sum_{i=1}^{N_S} \alpha_i y_i \Phi(s_i).\Phi(x) + b) = sgn(\sum_{i=1}^{N_S} \alpha_i y_i K(s_i, x) + b)$$

où s_i désignent les vecteurs supports.

Exemple de “mapping”



For $\vec{x} = (x_1, x_2)^\top \in \mathbb{R}^2$, let $\vec{\Phi}(\vec{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2) \in \mathbb{R}^3$:

$$\begin{aligned} K(\vec{x}, \vec{x}') &= x_1^2 x_1'^2 + 2x_1x_2x_1'x_2' + x_2^2 x_2'^2 \\ &= \left(x_1x_1' + x_2x_2'\right)^2 \\ &= \left(\vec{x}.\vec{x}'\right)^2 \ . \end{aligned}$$

Conditions de Mercer

- Si K est une fonction symétrique, il existe une fonction Φ telle que $K(x_i, x_j) = \Phi(x_i).\Phi(x_j)$ ssi : pour toute fonction f où $\int f(x_i)^2 dx$ est finie, alors on a $\int K(x_i, x_j) f(x_i) f(x_j) dx_i dx_j \geq 0$
- Si cette condition est vérifiée, alors on peut utiliser K dans le cadre des SVM.
- Mais cela ne nous dit toujours pas comment construire Φ.

Fonctions noyaux usuelles (1)

- **Polynomiale :** les polynômes de degré q ont pour fonction noyau associée : $K(x_i, x_j) = (x_i.x_j + c)^q$
- **RBF :** les fonction à base radiale ont pour fonction noyau associée : $K(x_i, x_j) = e^{-\gamma|x_i.x_j|^2}$
- **Gaussien :** il s'agit d'un cas particulier du noyau RBF avec pour fonction noyau : $K(x_i, x_j) = e^{-\frac{|x_i.x_j|^2}{2\sigma^2}}$
- **Sigmoïde :** inspiré de la fonction d'activation des réseaux de neurones, la fonction noyau est : $K(x_i, x_j) = \tanh(\gamma x^T.y + r)$

Fonctions noyaux usuelles (2)

On peut construire de nouvelles fonctions noyau à partir des fonctions noyaux “de base” (propriété de clôture) :

- $K(x, y) = K_1(x, y) + K_2(x, y)$
- $K(x, y) = a.K_1(x, y)$
- $K(x, y) = K_1(x, y) x K_2(x, y)$
- etc.

Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

Plan

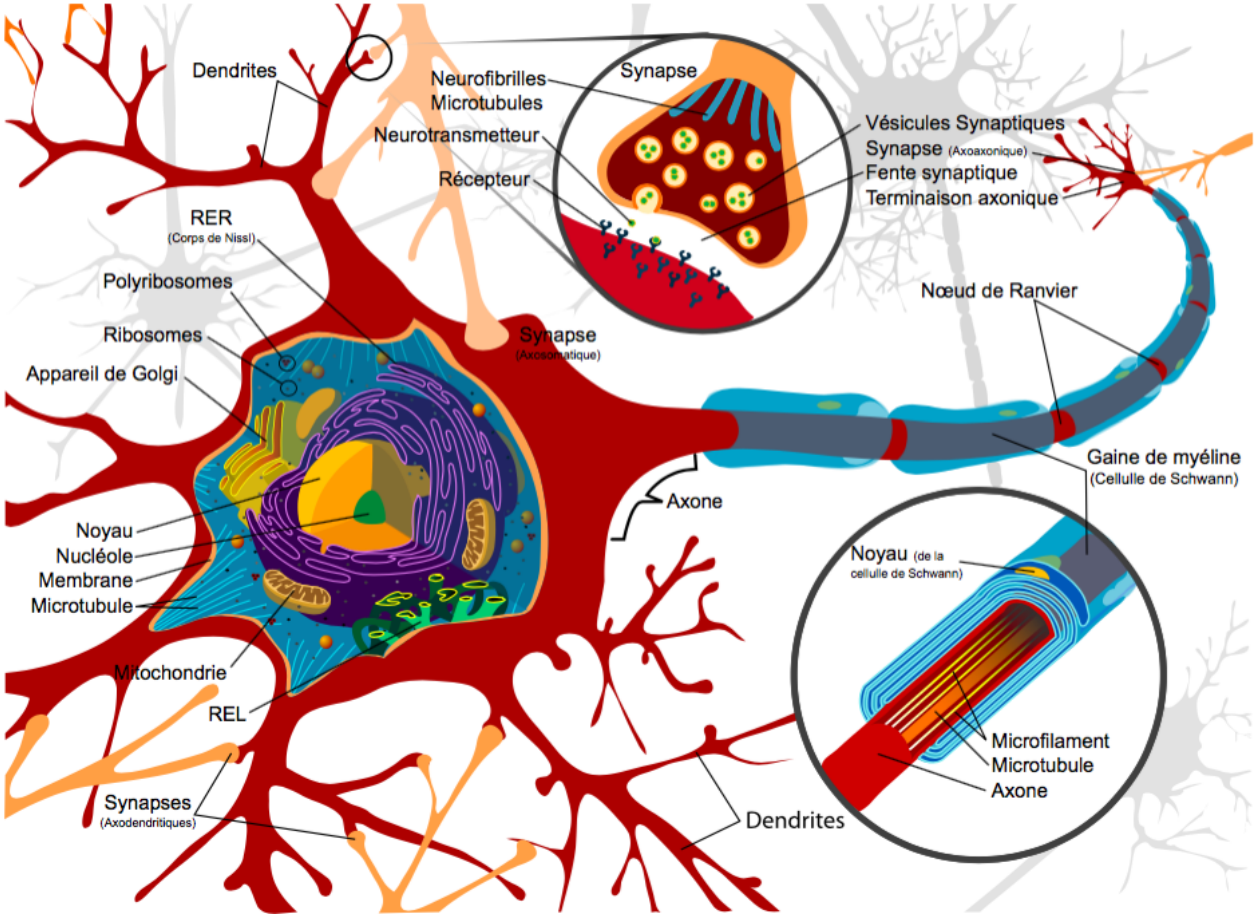
Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

Réseaux de neurones artificiels

- Modèles (lointainement) inspirés du fonctionnement de notre cerveau et de travaux en sciences cognitives et neurosciences
- Notre cerveau est capable de traiter des problèmes difficiles : reconnaissance de visages, de la parole, etc.
- Paradigme **connexionniste**, une vision *parallèle* et *distribuée* de l'activité du cerveau vu comme un système de traitement de l'information :
  - chaque neurone traite l'information indépendamment des autres avant d'en communiquer le résultat à d'autres neurones grâce aux synapses
  - des parties du cerveau (groupes de neurones) se spécialisent dans certaines tâches

Schéma d'un neurone biologique



Comparons les deux “machines”

Caractéristiques	Cerveau humain	Ordinateur
Support des données	**neurones**	circuits électroniques
Véhicule des données	neurotransmetteurs (substances chimiques)	impulsions électriques
Enregistrement des données	analogique (continu)	numérique (binaire)
Types de mémoires	à long terme à court terme registres sensoriels externes : bibliothèques	morte (ROM) vive (RAM) tampon (*buffer*) périphériques : disques, ...
Localisation, spécialisation des fonctions	aires cérébrales spécialisées (cortex visuel, ...)	circuits spécialisés (CPU, mémoires, contrôleurs, horloge, ...)
Nombre de cellules	≈ 30 000 000 000 (1010)	4 000 000 000 (109)
Liaisons par cellule	≈ 10 000 (104)	2 (cellules voisines)
Structure	**réseau** (non linéaire)	**liste** (linéaire)
Durée des impulsions	0,001 sec (10-3)	0,000 000 001 sec (10-9)
Vitesse de propagation	130 m/sec (102)	300 000 000 m/sec (108)
Temps d'accès (sec.)	0,1 sec (10-1)	0,000 000 1 sec (10-7)
Débit (*bits*/sec.)	faible (102)	fort (108)

http://intelligence-artificielle-tpe.e-monsite.com/pages/limites-technologiques-et-ethique-de-l-ia/cerveau-humain-et-robot.html

Un peu d'histoire

- 1943 : neurone formel de McCulloch & Pitts
  "A logical calculus of the ideas immanent in nervous activities"
- 1948 : Von Neumann – les réseaux d'automates
- 1949 : Donald Hebb – hypothèse de l'efficacité synaptique, notion de mémoire associative, premières règles locales d'apprentissage
- 1958 : Rosenblatt et Widrow - Perceptron et Adaline
- 1969 : Minsky et Papert – analyse théorique des capacités de calcul des réseaux à base de perceptron.
- Début des années 70 : stagnation des recherches sur les réseaux neuromimétiques ; report des efforts en Intelligence Artificielle symbolique (systèmes experts. . .)
- Années 70-80 : quelques avancées - perceptron multicouches (MLP), cartes auto-organisatrices (SOM), etc.
- Des mécanismes d'apprentissage performant pour le perceptron multicouches voient le jour (rétropropagation du gradient)

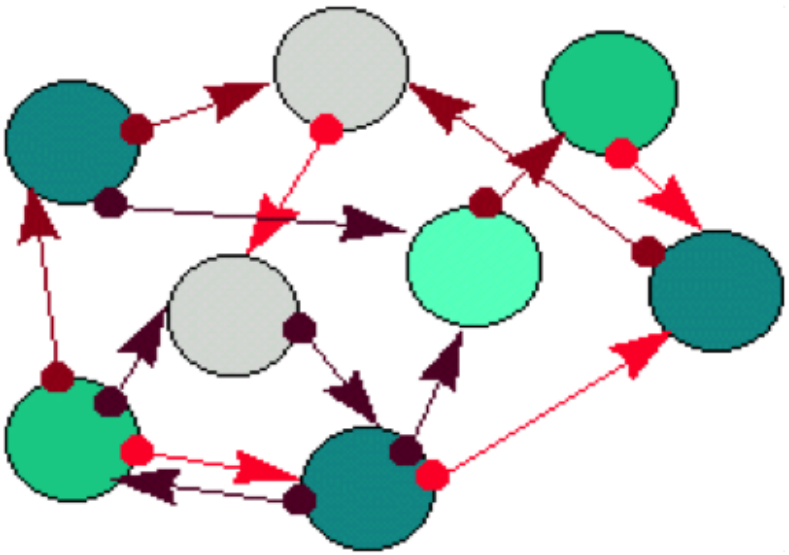
Un peu d'histoire récente

Avènement de l'apprentissage profond (*deep learning*) :

- avant 2000 : de nombreux travaux importants mais qui passent inaperçus (la faute aux SVM ?), tels que les machines de Boltzmann (1985) ou la mémoire à *long short-term memory* (LSTM) pour les réseaux récurrents (1997)
- Peu à peu, des progrès importants sont faits quant à la puissance de calculs (ex. GPUs) et les architectures profonde gagnent les compétitions
- 2013 : un réseau de neurones remporte la compétition ImageNet
- 2015 : Google annonce des taux de reconnaissance de visage de l'ordre de 99,63% avec FaceNet (réseau de neurones à 22 couches !)
- octobre 2015 : victoire d'alphaGo contre le champion européen de Go, Fan Hui
- mi-mars 2016 : victoire de DeepMind contre le champion du monde de Go, Lee Sedol (4 parties à 1)

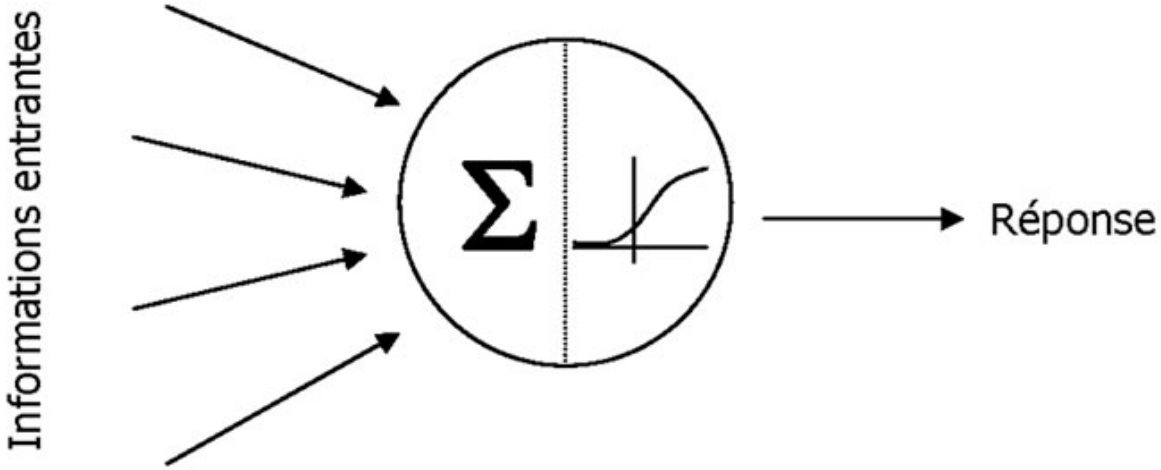
S’inspirer des modèles naturels

Un **réseau neuromimétique** est un graphe pondéré orienté dont les sommets sont appelés “neurones’ et sont dotés d’un comportement d’automate simple.



Modèle de neurone artificiel

Figure 1 – Neurone artificiel (McCullogh et Pitts, 1943)



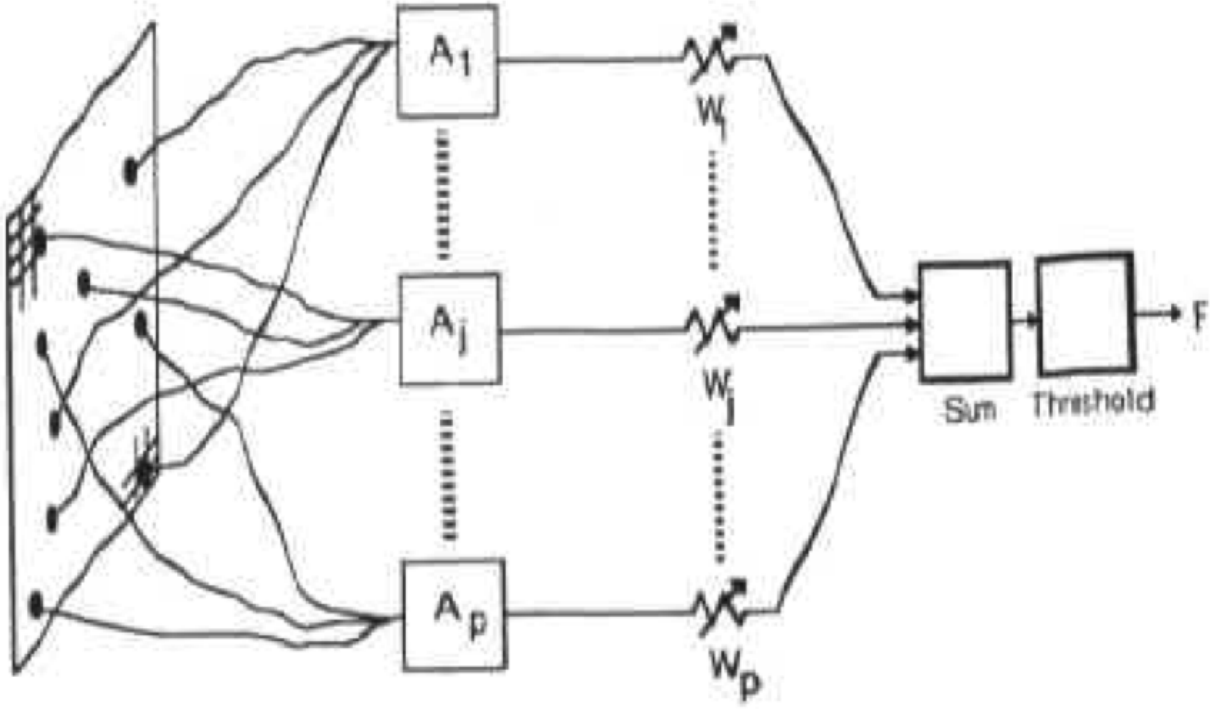
Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

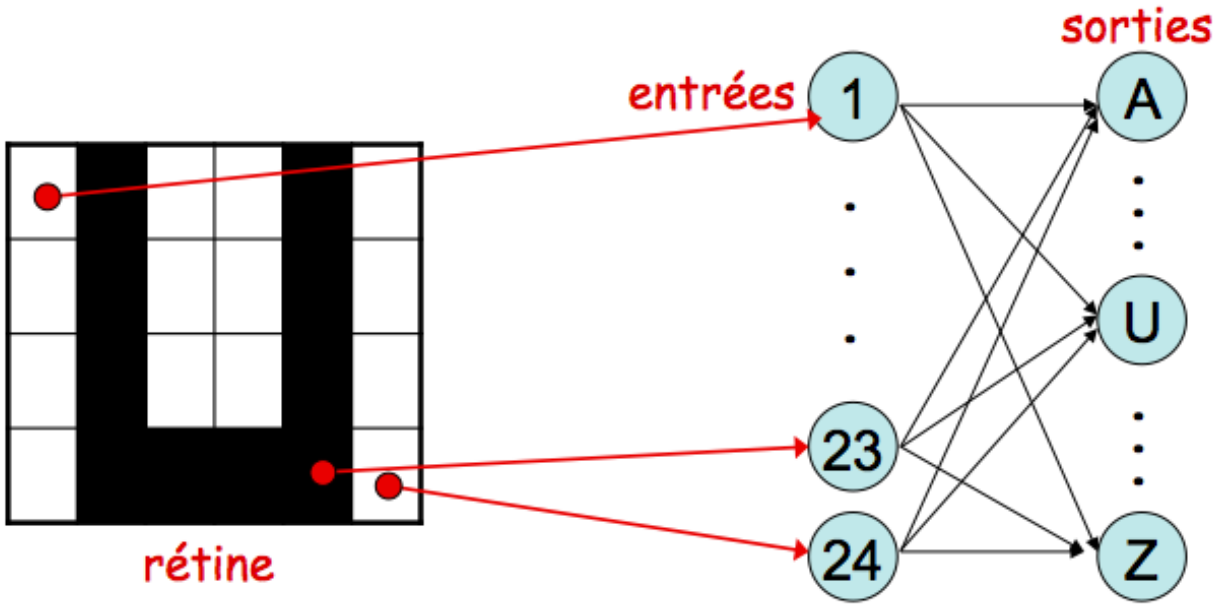
Le perceptron de Rosenblatt (1957)

Architecture simple : une entrée (la rétine) et une sortie.

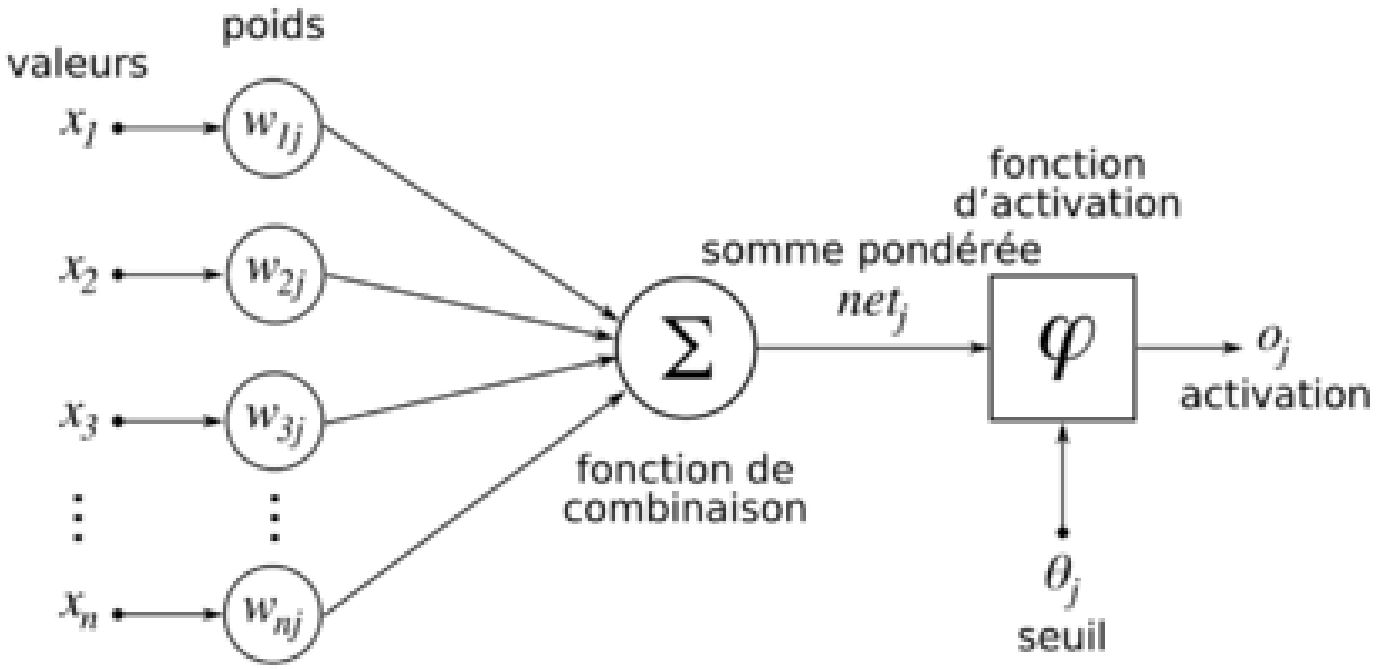


Exemple : associer des formes à des classes

Rétine de 4 x 6 éléments (image d'une lettre) avec 26 éléments en sortie



Le Perceptron



Le Perceptron

Cas à un seul neurone de sortie

Modèle de McCulloch et Pitts :

- Soit $\mathbf{x} = (x_1, \ldots, x_p)$ dans $\mathbb{R}^p$ le signal d'entrée
- Chaque x_i est associé à un poids w_i
- Le vecteur $\mathbf{w}$ est appelé coefficients synaptiques
- On ajoute une constante w_0 appelée le biais

Le Perceptron

Cas à un seul neurone de sortie

Modèle de McCulloch et Pitts :

- Soit $\mathbf{x} = (x_1, \ldots, x_p)$ dans $\mathbb{R}^p$ le signal d'entrée
- Chaque x_i est associé à un poids w_i
- Le vecteur $\mathbf{w}$ est appelé coefficients synaptiques
- On ajoute une constante w_0 appelée le biais

Signal post-synaptique

$s(x) = w_0 + \sum_{i=1}^{p} (w_i \times x_i)$

Le Perceptron

Cas à un seul neurone de sortie

Modèle de McCulloch et Pitts :

- Soit $\mathbf{x} = (x_1, \ldots, x_p)$ dans $\mathbb{R}^p$ le signal d'entrée
- Chaque x_i est associé à un poids w_i
- Le vecteur $\mathbf{w}$ est appelé coefficients synaptiques
- On ajoute une constante w_0 appelée le biais

Signal post-synaptique

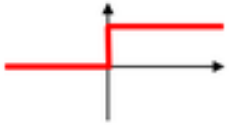
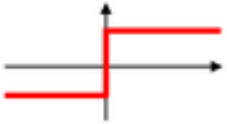
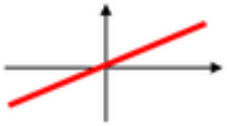
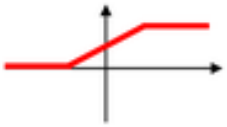
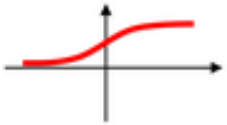
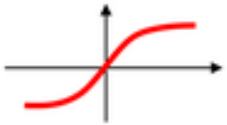
$s(x) = w_0 + \sum_{i=1}^{p} (w_i \times x_i)$

ou, si $\mathbf{x} = (1, x_1, \ldots, x_p)$: $s(x) = \sum_{i=0}^{p} w_i x_i = \mathbf{w}.\mathbf{x}$

Fonction d'activation

$\hat{y}_{\mathbf{w}}(\mathbf{x}) = h(s(\mathbf{x}))$ où h peut être définie de différentes manières

Exemples de fonctions d'activation

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer NN	

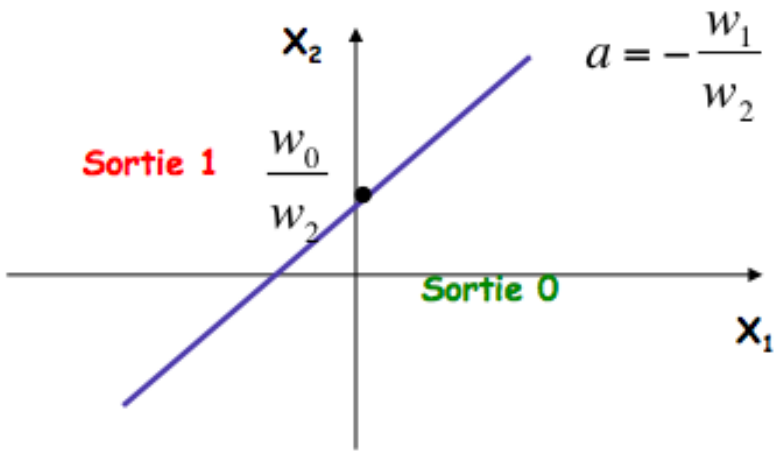
Neurone à deux entrées

Règle d'activation linéaire

Si $s = w_1.x_1 + w_2.x_2 > w_0$ alors $\hat{y}$=1
Si $s = w_1.x_1 + w_2.x_2 \leq w_0$ alors $\hat{y}$=0

Géométriquement

Cela signifie qu'on a divisé le plan en deux régions par une droite d'équation $w_1.x_1 + w_2.x_2 - w_0 = 0$



Lien avec la régression logistique

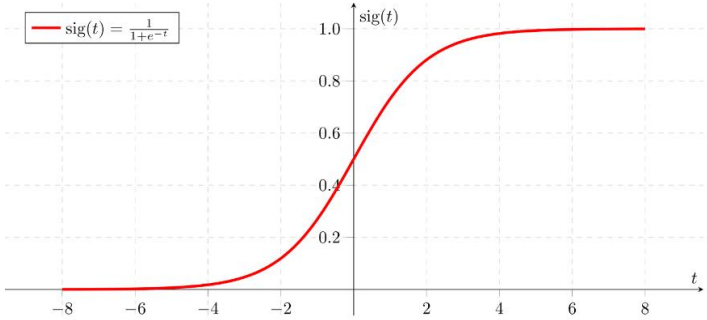
Principe

- on cherche à modéliser de façon linéaire $\ln \frac{P(Y=1|\mathbf{X}=\mathbf{x})}{1-P(Y=1|\mathbf{X}=\mathbf{x})}$:

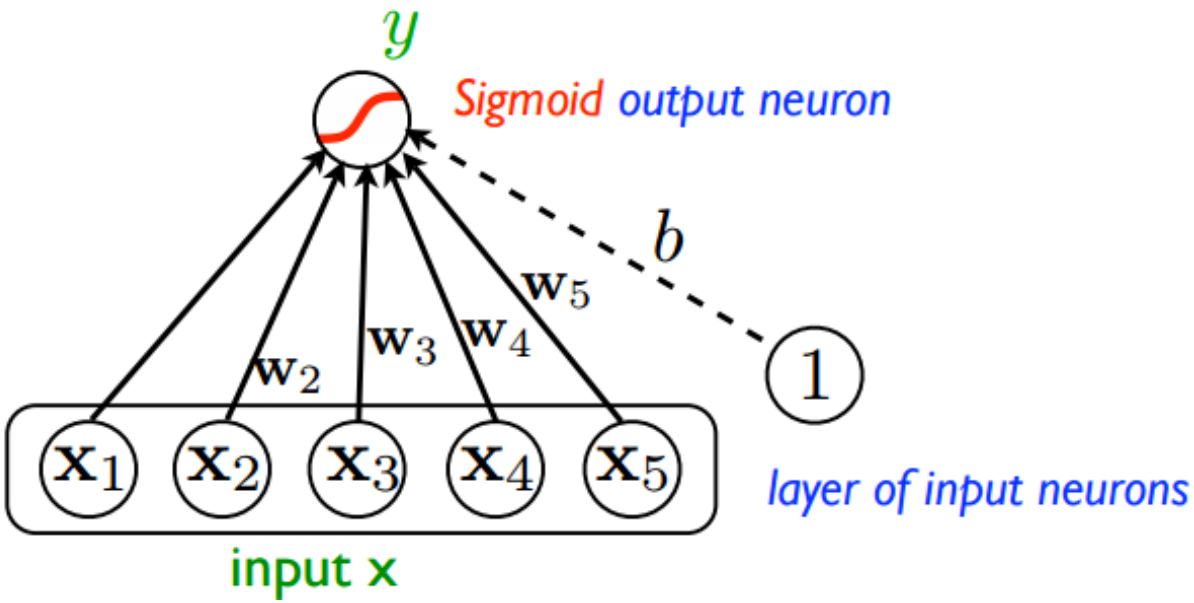
$$\ln \frac{P(Y=1|\mathbf{X}=\mathbf{x})}{1-P(Y=1|\mathbf{X}=\mathbf{x})} = \underbrace{\ln \frac{\pi(\mathbf{x})}{1-\pi(\mathbf{x})}}_{logit(\pi(\mathbf{x}))} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

ce modèle s'écrit aussi

$$\pi(\mathbf{x}) = \frac{\exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right)}{1 + \exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right)}$$

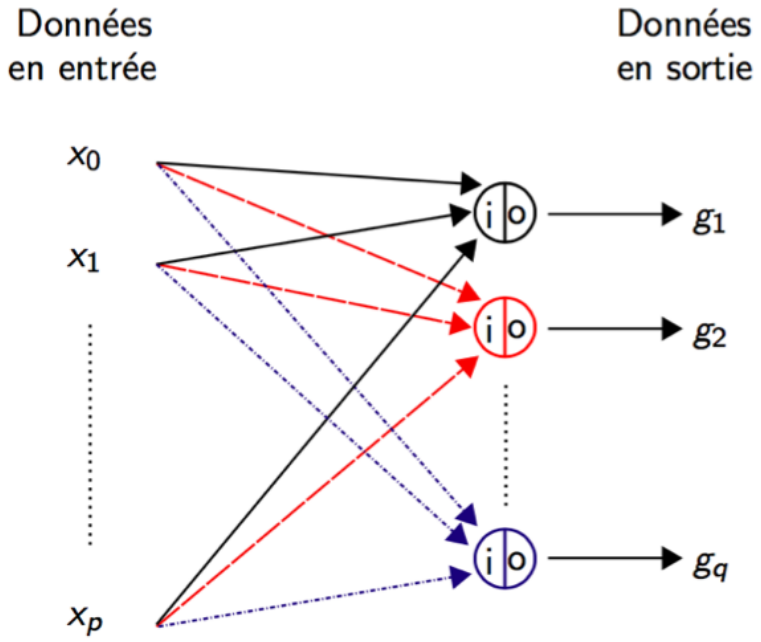


Une autre représentation bien utile



Remarques sur le perceptron

- Sans couche cachée, on retrouve les modèles linéaires
- Avec la fonction sigmoïd, il s'agit d'une régression logistique et on interprète les sorties comme des probabilités
- Pour le problème multiclasse, on traite q problèmes binaires indépendants :



A nouveau : relations entre LR et ANN

Pour la catégorisation en $q = 2$ classes, on utilise un seul perceptron de sortie et la fonction sigmoïde :

$$g(x) = h(s(x)) = \frac{1}{1 + exp(-w^T.x)}$$

La régression logistique polytomique est équivalente à un ANN avec q perceptrons en parallèle et des fonctions d'activation softmax :

$$g_k(x) = h(\mathbf{w}_k^T.\mathbf{x}) = \frac{exp(\mathbf{w}_k^T.\mathbf{x})}{\sum_j exp(\mathbf{w}_j^T.\mathbf{x})}$$

Le softmax permet de garantir une probabilité en sortie.

Cas de la fonction booléenne “AND”

X^1	X^2	AND
1	1	1
1	0	0
0	1	0
0	0	0

Cas de la fonction booléenne “AND”

X^1	X^2	AND
1	1	1
1	0	0
0	1	0
0	0	0

Signal post-synaptique et fonction d’Heaviside :

$$w_0 + w_1 X^1 + w_2 X^2 \lesseqgtr 0$$

Cas de la fonction booléenne “AND”

X^1	X^2	AND
1	1	1
1	0	0
0	1	0
0	0	0

Signal post-synaptique et fonction d’Heaviside :

$$w_0 + w_1 X^1 + w_2 X^2 \underset{>}{\leq} 0$$

$$\begin{cases} w_0 + w_1 + w_2 > 0 \\ w_0 + w_1 \leq 0 \\ w_0 + w_2 \leq 0 \\ w_0 \leq 0 \end{cases}$$

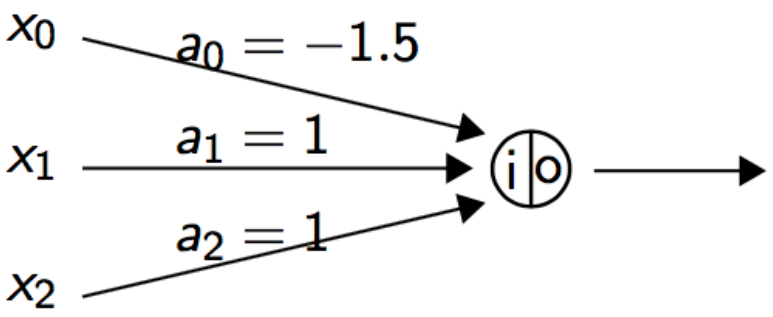
Cas de la fonction booléenne “AND”

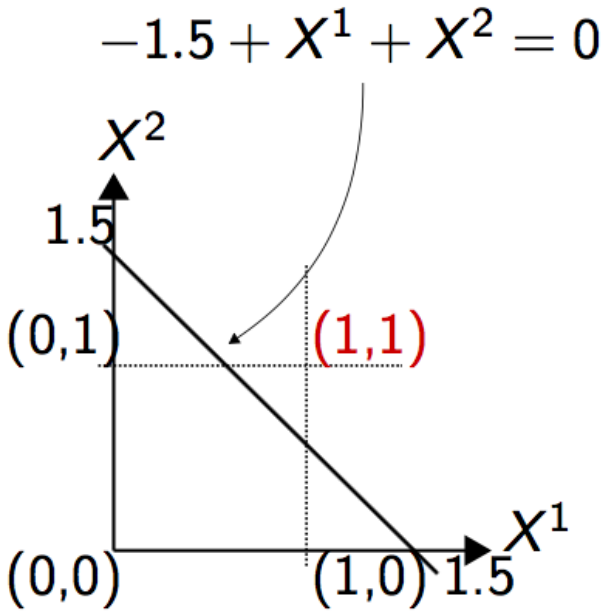
X^1	X^2	AND
1	1	1
1	0	0
0	1	0
0	0	0

Signal post-synaptique et fonction d’Heaviside :

$$w_0 + w_1 X^1 + w_2 X^2 \lesseqgtr 0$$

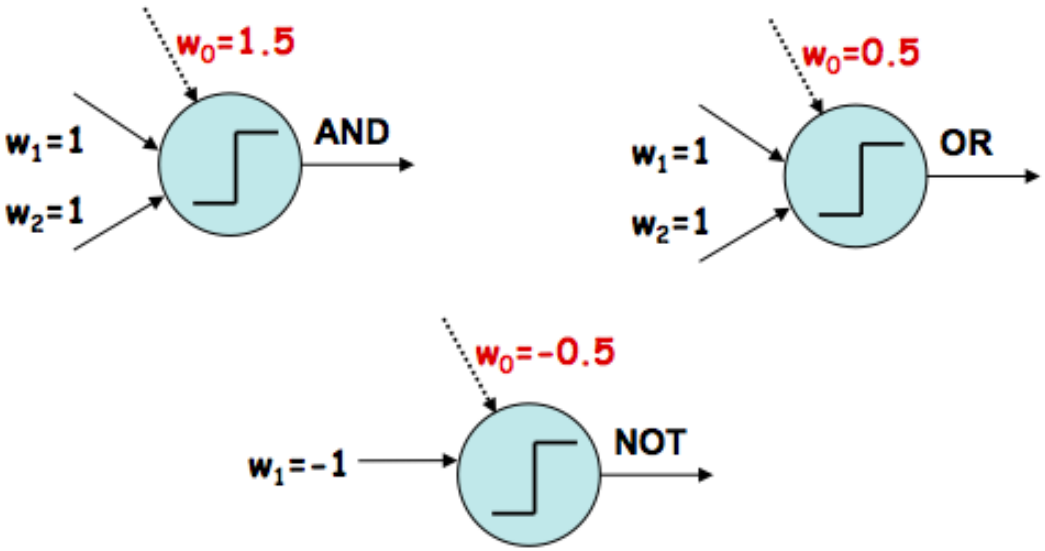
$$\begin{cases} w_0 + w_1 + w_2 > 0 \\ w_0 + w_1 \leq 0 \\ w_0 + w_2 \leq 0 \\ w_0 \leq 0 \end{cases}$$





Note au sujet du biais w_0

L'une des motivations initiales était de pouvoir représenter des fonctions logiques :



Cas de la fonction booléenne “XOR”

X^1	X^2	XOR
1	1	0
1	0	1
0	1	1
0	0	0

Cas de la fonction booléenne “XOR”

X^1	X^2	XOR
1	1	0
1	0	1
0	1	1
0	0	0

Signal post-synaptique et fonction d’Heaviside :

$$w_0 + w_1 X^1 + w_2 X^2 \lesseqgtr 0$$

Cas de la fonction booléenne “XOR”

X^1	X^2	XOR
1	1	0
1	0	1
0	1	1
0	0	0

Signal post-synaptique et fonction d’Heaviside :

$$w_0 + w_1 X^1 + w_2 X^2 \underset{>}{\leq} 0$$

$$\begin{cases} w_0 + w_1 + w_2 \leq 0 \\ w_0 + w_1 > 0 \\ w_0 + w_2 > 0 \\ w_0 \leq 0 \end{cases}$$

Cas de la fonction booléenne “XOR”

X^1	X^2	XOR
1	1	0
1	0	1
0	1	1
0	0	0

Signal post-synaptique et fonction d’Heaviside :

$$w_0 + w_1 X^1 + w_2 X^2 \underset{>}{\leq} 0$$

$$\begin{cases} w_0 + w_1 + w_2 \leq 0 \\ w_0 + w_1 > 0 \\ w_0 + w_2 > 0 \\ w_0 \leq 0 \end{cases}$$

Ensemble de contraintes **incompatibles**.

Cas de la fonction booléenne “XOR”

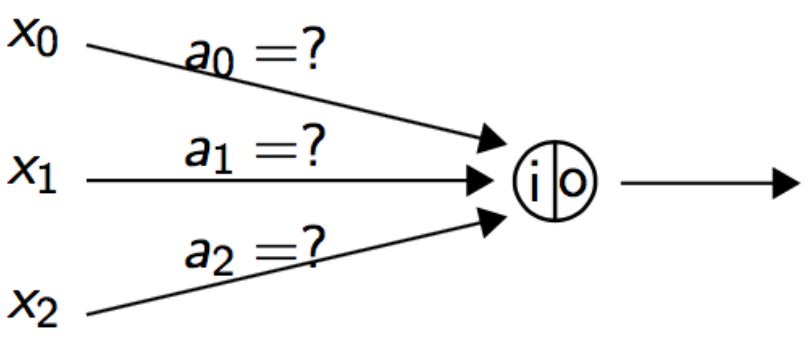
X^1	X^2	XOR
1	1	0
1	0	1
0	1	1
0	0	0

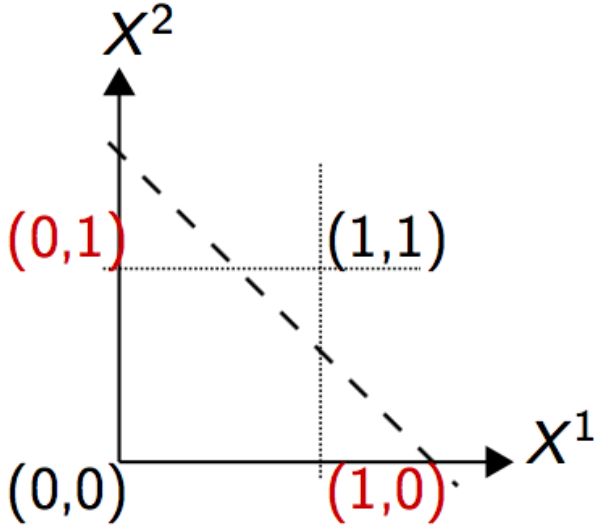
Signal post-synaptique et fonction d’Heaviside :

$$w_0 + w_1 X^1 + w_2 X^2 \lesseqgtr 0$$

$$\begin{cases} w_0 + w_1 + w_2 \leq 0 \\ w_0 + w_1 > 0 \\ w_0 + w_2 > 0 \\ w_0 \leq 0 \end{cases}$$

Ensemble de contraintes **incompatibles**.



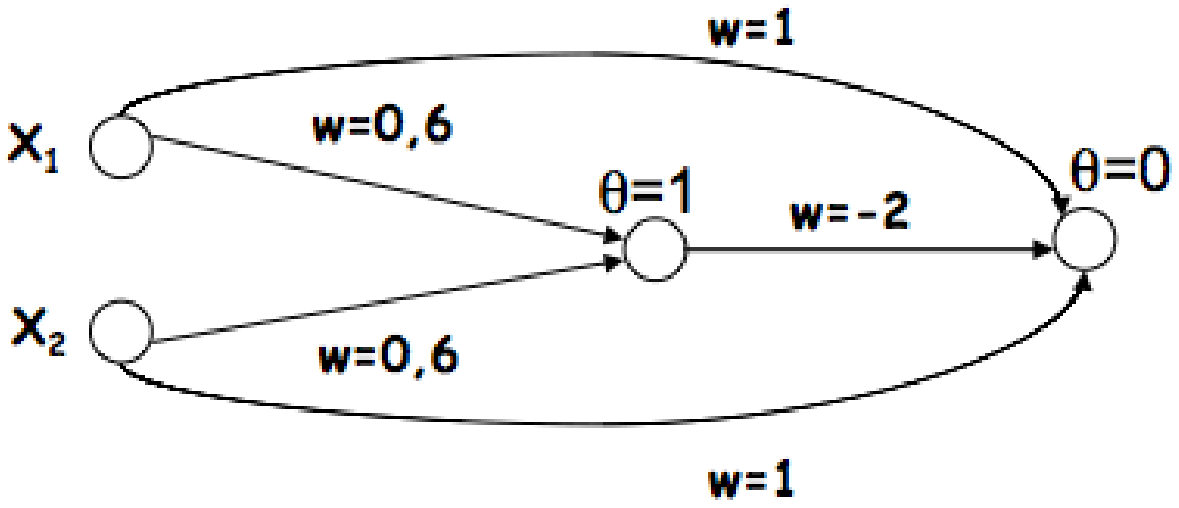


Et pourtant, une solution existe...

On peut apprendre le XOR avec un réseau à une couche cachée :

Et pourtant, une solution existe...

On peut apprendre le XOR avec un réseau à une couche cachée :



Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

Le perceptron multicouches

- Les modèles précédents définissent des modèles linéaires avec certaines limites.

Le perceptron multicouches

- Les modèles précédents définissent des modèles linéaires avec certaines limites.
- Le perceptron multicouche (“multilayer perceptron”) est une généralisation de ces modèles :

Le perceptron multicouches

- Les modèles précédents définissent des modèles linéaires avec certaines limites.
- Le perceptron multicouche (“multilayer perceptron”) est une généralisation de ces modèles :
  - en régression il permet de traiter les cas **non linéaires** de régression

Le perceptron multicouches

- Les modèles précédents définissent des modèles linéaires avec certaines limites.
- Le perceptron multicouche (“multilayer perceptron”) est une généralisation de ces modèles :
  - en régression il permet de traiter les cas **non linéaires** de régression
  - en classification, il permet de déterminer des fonctions de décision **non linéaire** permettant de résoudre le problème “XOR” précédent par exemple

Le perceptron multicouches

- Les modèles précédents définissent des modèles linéaires avec certaines limites.
- Le perceptron multicouche (“multilayer perceptron”) est une généralisation de ces modèles :
  - en régression il permet de traiter les cas **non linéaires** de régression
  - en classification, il permet de déterminer des fonctions de décision **non linéaire** permettant de résoudre le problème “XOR” précédent par exemple
- Il consiste en l’ajout de **couches de neurones dites cachées** entre les données en entrée et les données en sortie.

Pourquoi considérer des fonctions non linéaires ?

$$y = W_2(W_1.x + b_1) + b_2$$

Pourquoi considérer des fonctions non linéaires ?

$$y = W_2(W_1.x + b_1) + b_2$$

$$y = W_2.W_1.x + W_2.b_1 + b_2$$

Pourquoi considérer des fonctions non linéaires ?

$$y = W_2(W_1.x + b_1) + b_2$$

$$y = W_2.W_1.x + W_2.b_1 + b_2$$

$$y = W.x + b$$

Pourquoi considérer des fonctions non linéaires ?

$$y = W_2(W_1.x + b_1) + b_2$$

$$y = W_2.W_1.x + W_2.b_1 + b_2$$

$$y = W.x + b$$

En conclusion

Le réseau à 2 couches est équivalent à un réseau à 1 couche linéaire

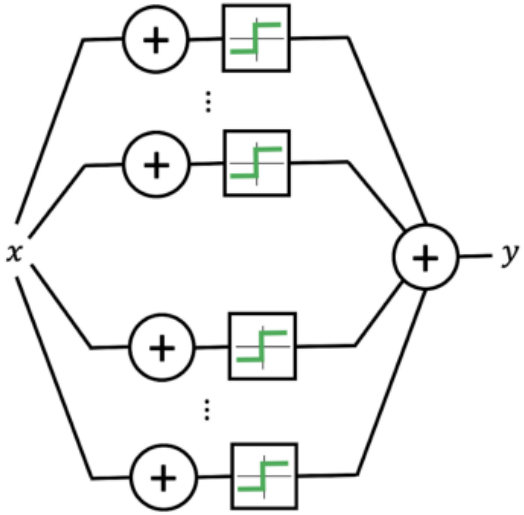
Approximation universelle

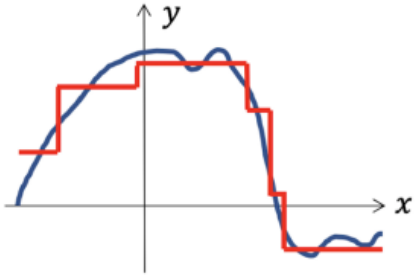
Théorème d'approximation universelle

Toute fonction continue (vérifiant certaines propriétés) peut être approximée via un MLP avec au moins une couche cachée contenant suffisamment de neurones et des fonctions d'activation non linéaires (ex. sigmoïde, mais aussi reLU).

Attention : dire qu'un MLP a la "capacité" de représenter toute fonction continue ne signifie pas que l'on va être capable d' *apprendre* les paramètres du réseau de manière optimale.

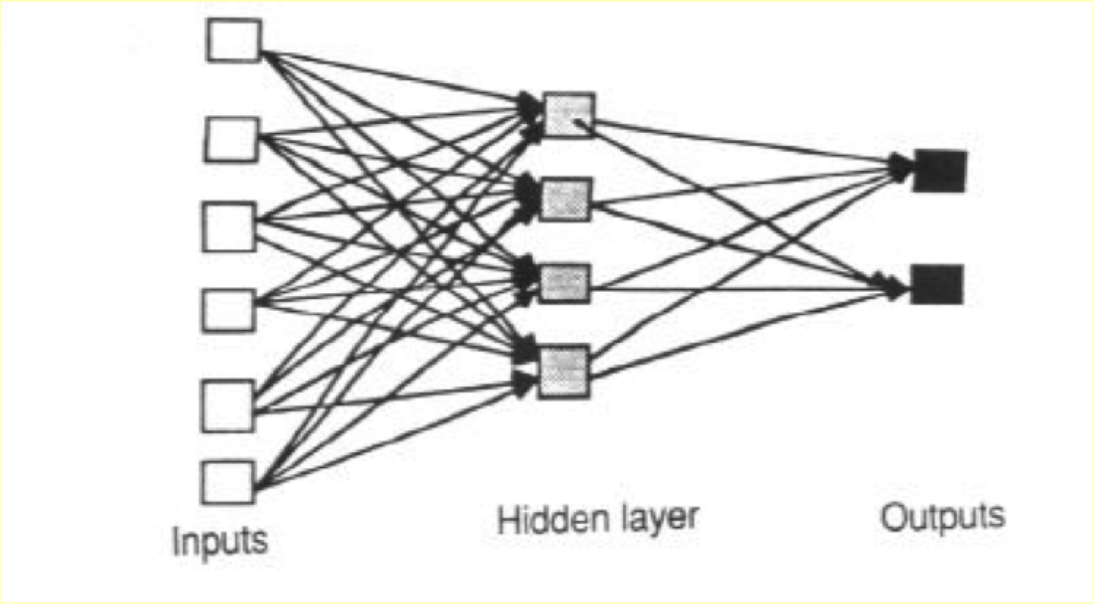
Approximation universelle



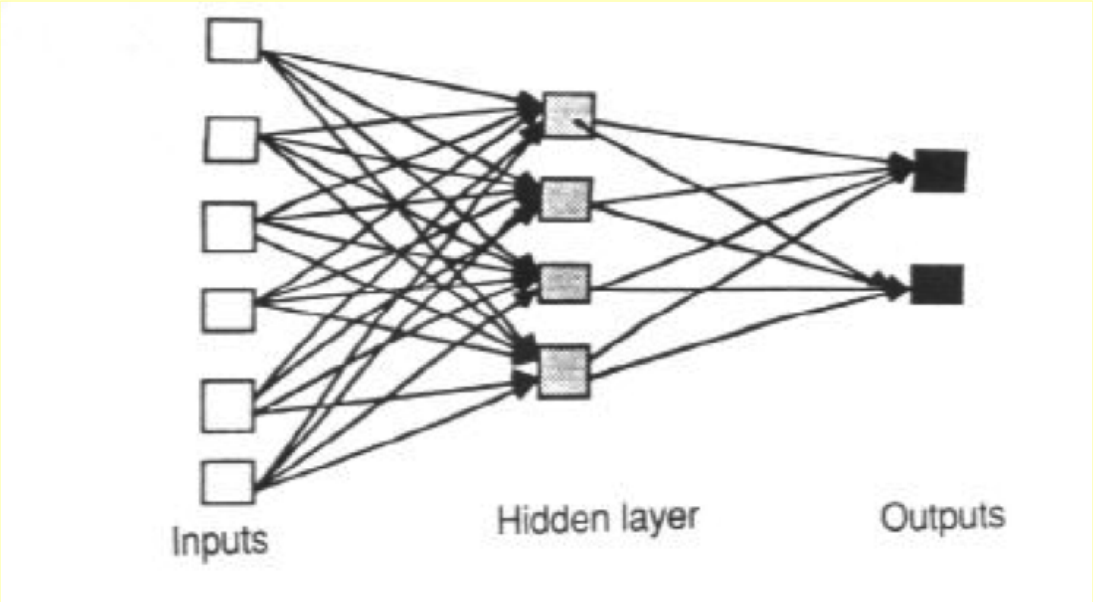


(tiré de : Geometric Deep Learning, https://arxiv.org/pdf/2104.13478)

Le perceptron multicouches

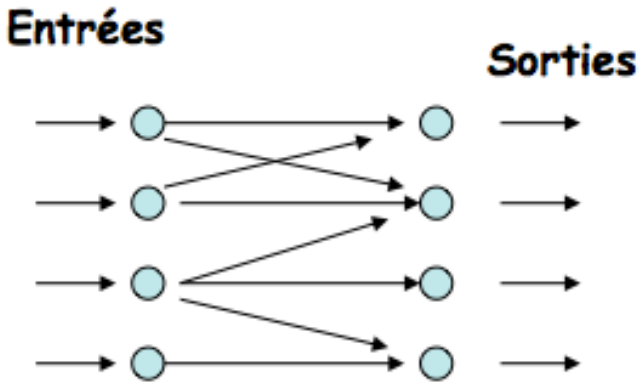


Le perceptron multicouches

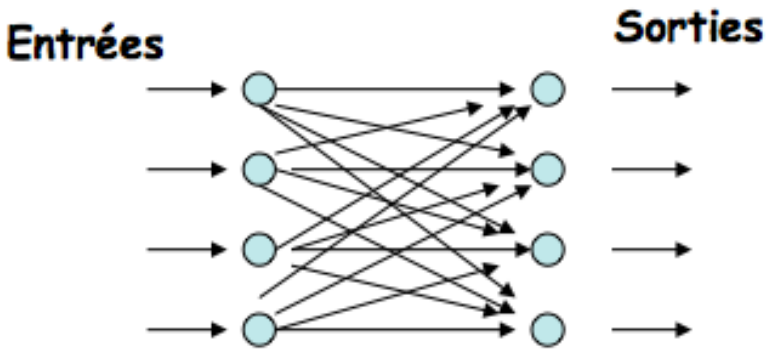


Petite démo avec TensorFlow :
`http://playground.tensorflow.org`

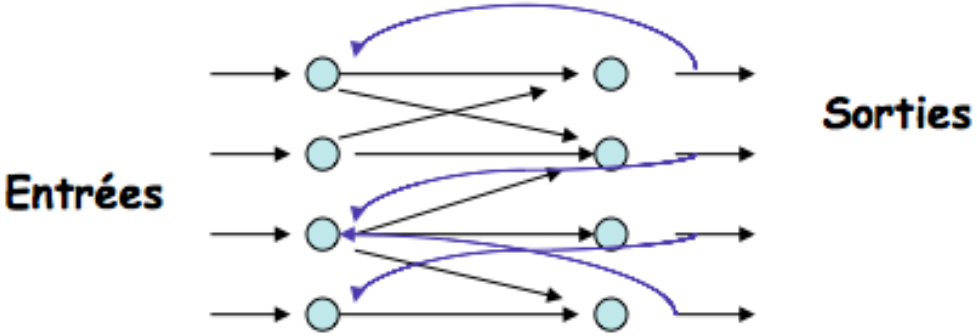
Architecture des réseaux de neurones



Connexions directes
(*feed-forward*)



Connexions directes - complet



Connexions récurrentes

Perceptron - Apprentissage des poids

Loi de Hebb

Lorsque deux neurones sont excités conjointement, il se crée ou renforce un lien les unissant : $w_i' = w_i + \eta(\hat{y}.x_i)$

Perceptron - Apprentissage des poids

Loi de Hebb

Lorsque deux neurones sont excités conjointement, il se crée ou renforce un lien les unissant : $w_i' = w_i + \eta(\hat{y}.x_i)$

Cette loi a inspiré le perceptron de Rosenblatt qui l'adapte pour prendre en compte l'erreur observée :

$$w_i' = w_i + \eta(y - \hat{y})x_i$$

Perceptron - Apprentissage des poids

Loi de Hebb

Lorsque deux neurones sont excités conjointement, il se crée ou renforce un lien les unissant : $w_i' = w_i + \eta(\hat{y}.x_i)$

Cette loi a inspiré le perceptron de Rosenblatt qui l'adapte pour prendre en compte l'erreur observée :

$$w_i' = w_i + \eta(y - \hat{y})x_i$$

Dans le perceptron de Rosenblatt, la règle passe par la fonction d'activation : $\hat{y} = h(\sum_{i=0}^{n} w_i x_i)$

Avec Adaline (1960), la règle de Widrow-Hoff utilise directement la somme pondérée des entrées : $\hat{y} = \sum_{i=0}^{n} w_i x_i$

Schéma général de l'algorithme pour un neurone

Pour chaque observation :

Schéma général de l'algorithme pour un neurone

Pour chaque observation :

- Phase de **propagation**
  - calculer l'activation du neurone
  - calculer la sortie de la fonction choisie

Schéma général de l'algorithme pour un neurone

Pour chaque observation :

- Phase de **propagation**
  - calculer l'activation du neurone
  - calculer la sortie de la fonction choisie
- Calcul de l'**erreur**

Schéma général de l'algorithme pour un neurone

Pour chaque observation :

- Phase de **propagation**
  - calculer l'activation du neurone
  - calculer la sortie de la fonction choisie
- Calcul de l'**erreur**
- **Mise à jour** des poids pour réduire l'erreur attendue

Schéma général de l'algorithme pour un neurone

Pour chaque observation :

- Phase de **propagation**
  - calculer l'activation du neurone
  - calculer la sortie de la fonction choisie
- Calcul de l'**erreur**
- **Mise à jour** des poids pour réduire l'erreur attendue
  Dans le cas d'une fonction simple à seuil (*heaviside*), les règles sont simples :
  - $w_{t+1} = w_t + x$ si y est positif et $\hat{y} = w_t.x \leq 0$
  - $w_{t+1} = w_t - x$ si y est négatif et $\hat{y} = w_t.x > 0$

Illustration de l'apprentissage

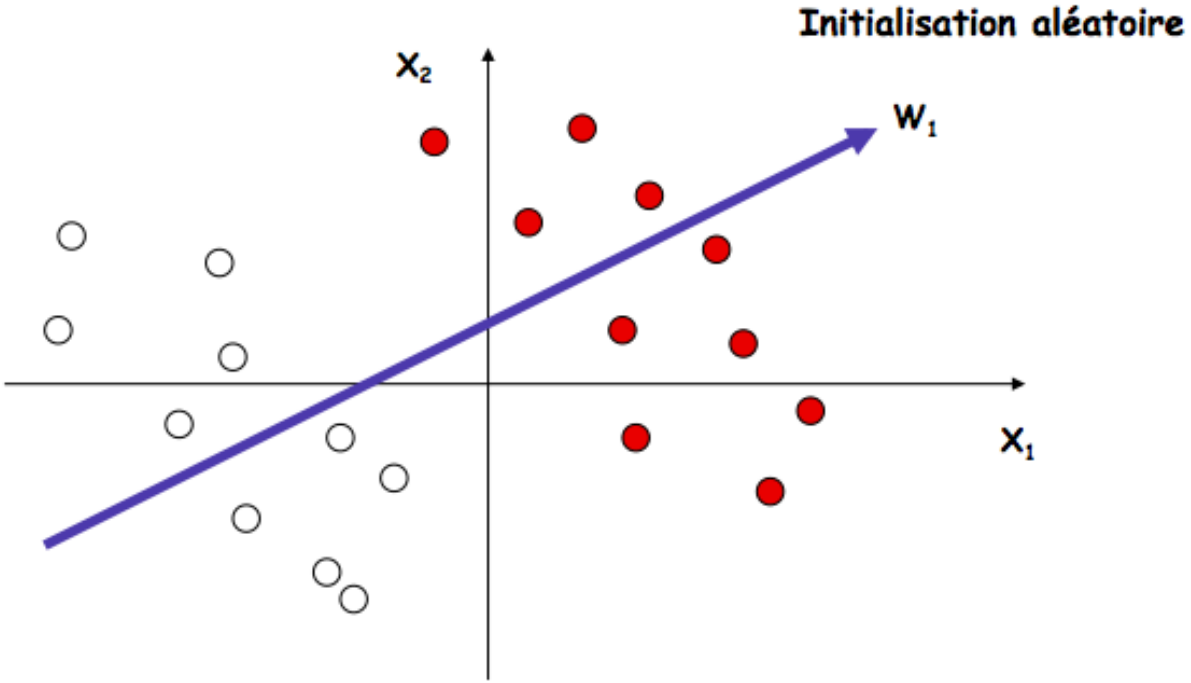


Illustration de l'apprentissage

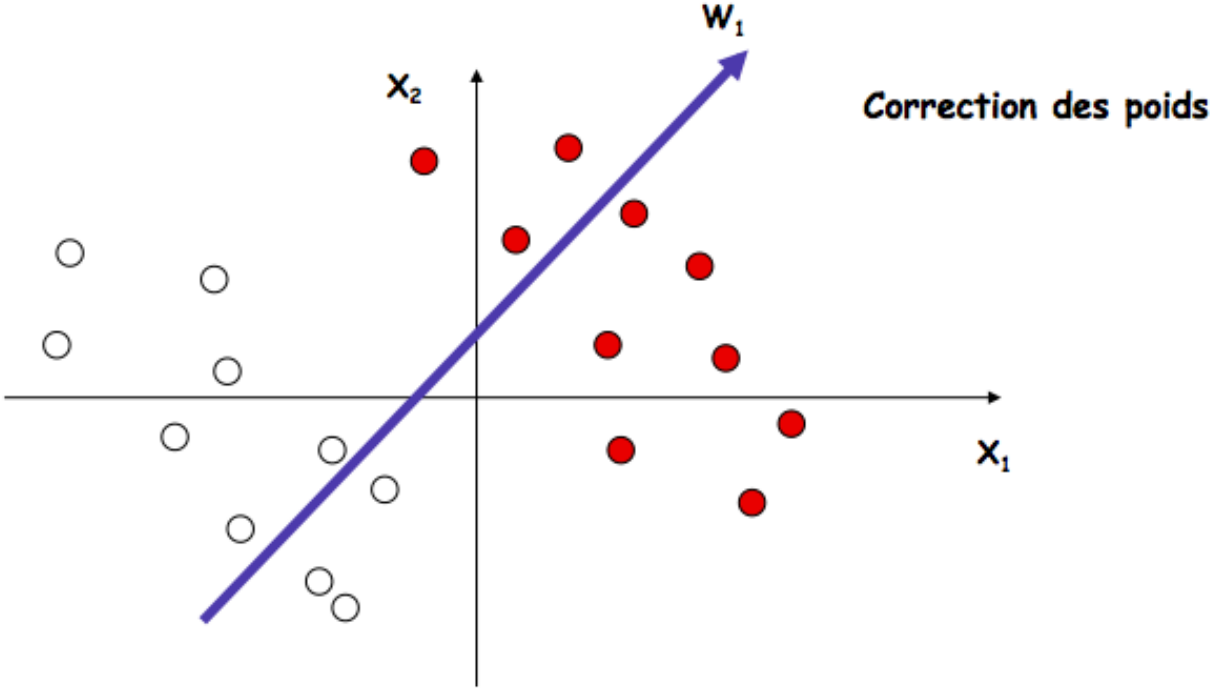


Illustration de l'apprentissage

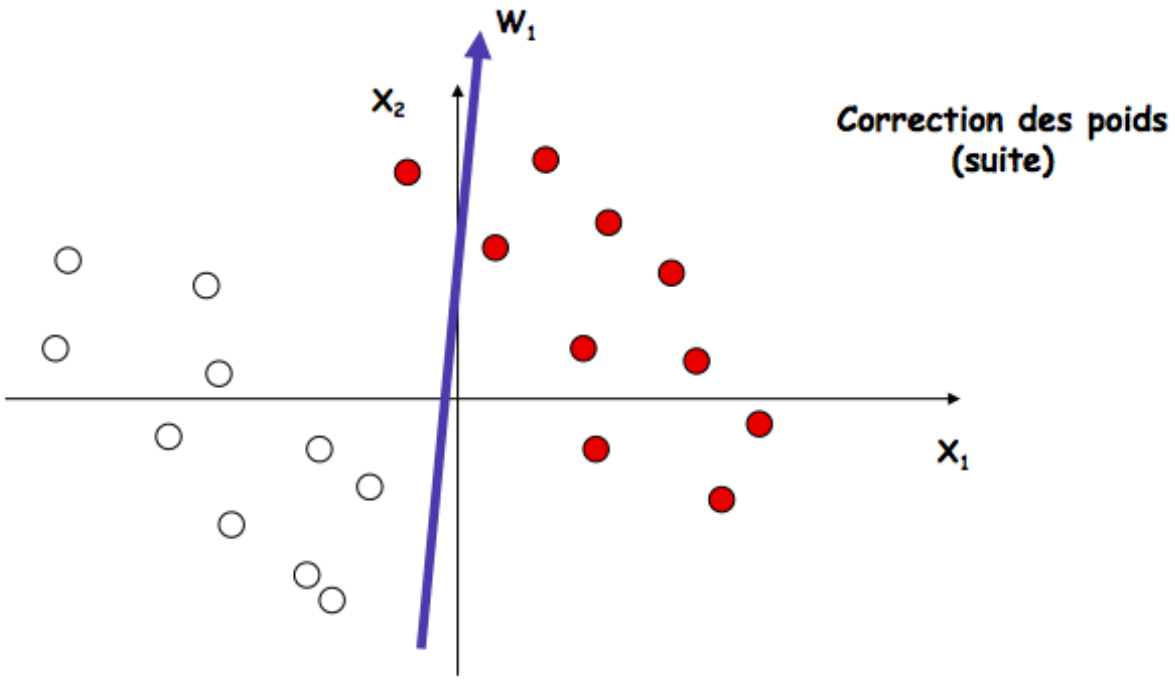
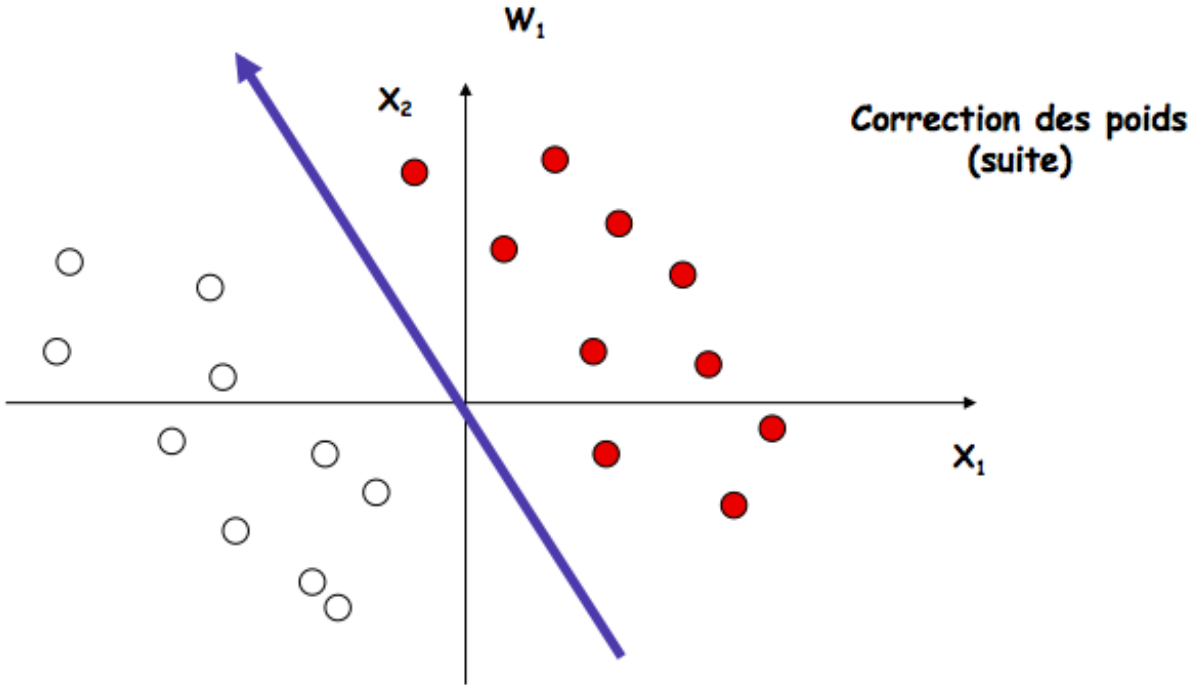
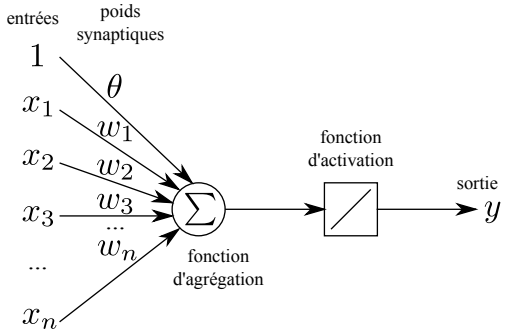


Illustration de l'apprentissage



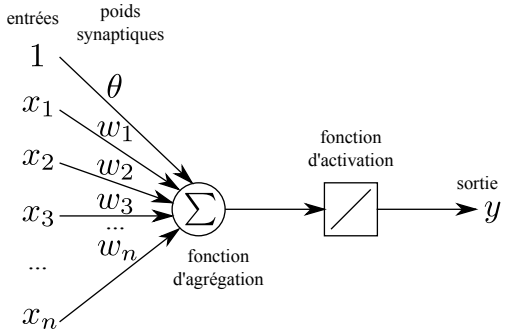
Perceptron - Apprentissage des poids

<u>Sortie du neurone</u>: $\hat{y} = \sum_{i=0}^{n} w_i x_i$



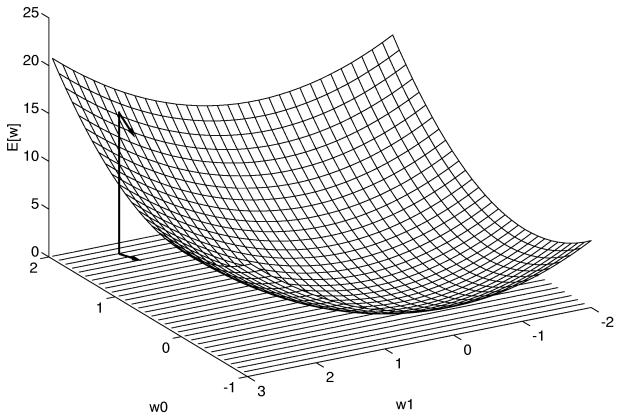
Perceptron - Apprentissage des poids

Sortie du neurone: $\hat{y} = \sum_{i=0}^{n} w_i x_i$

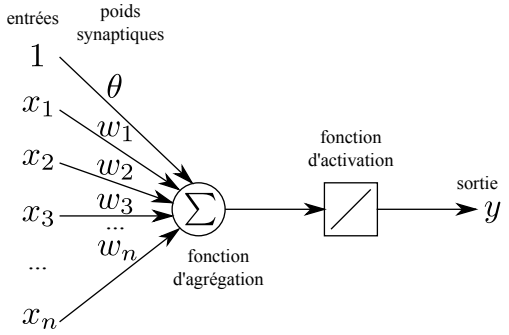


Apprentissage: On veut minimiser l'erreur

quadratique $E = \frac{1}{2} \sum_{\mathbf{x}} (y - \hat{y})^2$



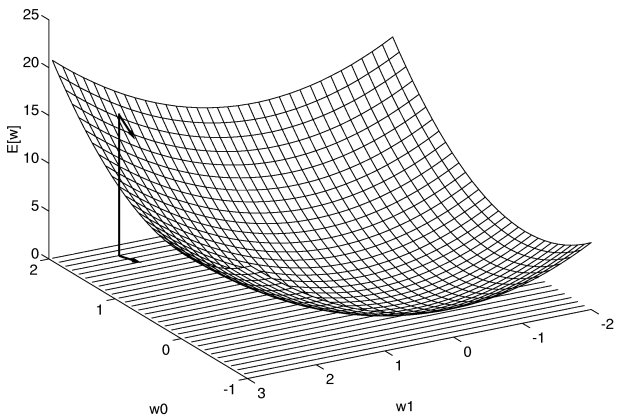
Perceptron - Apprentissage des poids



Sortie du neurone: $\hat{y} = \sum_{i=0}^{n} w_i x_i$

Apprentissage: On veut minimiser l'erreur

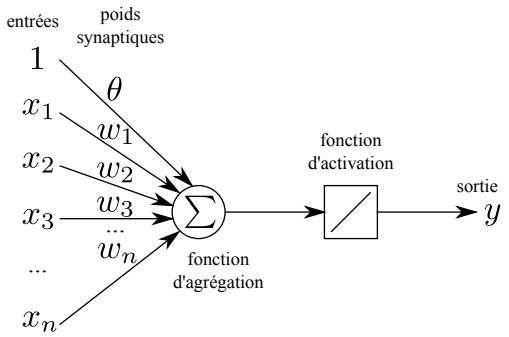
quadratique $E = \frac{1}{2} \sum_{\mathbf{x}} (y - \hat{y})^2$



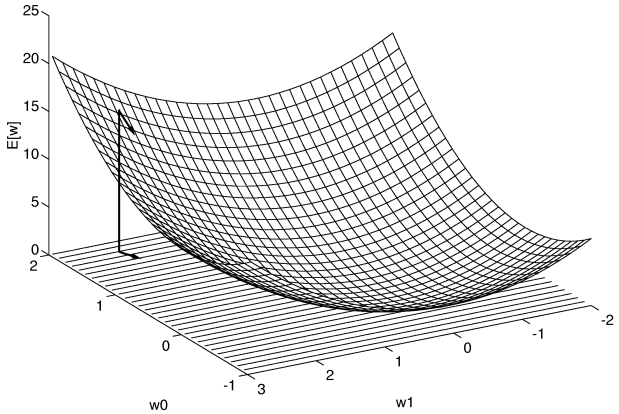
$$
\begin{aligned}
\Delta w_i &= -\eta \frac{\partial E}{\partial w_i} \\
&= -\frac{\eta}{2} \sum_{\mathbf{x}} \frac{\partial (y - \hat{y})^2}{\partial w_i} \\
&= -\frac{\eta}{2} \sum_{\mathbf{x}} -2 \frac{\partial \hat{y}}{\partial w_i} (y - \hat{y}) = \sum_{\mathbf{x}} \eta x_i (y - \hat{y})
\end{aligned}
$$

Perceptron - Apprentissage des poids

Sortie du neurone: $\hat{y} = \sum_{i=0}^{n} w_i x_i$



Apprentissage: On veut minimiser l'erreur quadratique $E = \frac{1}{2} \sum_{\mathbf{x}} (y - \hat{y})^2$ par descente de gradient stochastique (un seul exemple **x** utilisé à la fois) :
$\Delta \mathbf{w} = \eta \mathbf{x}(y - \hat{y})$



Optimisation de E par descente du gradient stochastique :
$\Delta \mathbf{w} = \eta \mathbf{x}(y - \hat{y})$

Propriétés

- Classifieur linéaire
- Convergence garantie si η est faible (même si les données ne sont pas linéairement séparables)
- Convergence efficace (car la fonction à optimiser est quadratique)
- Convergence souvent plus rapide en mode en ligne mais pas garantie comme en mode *batch*

Pseudo-code de l'algorithme

Require: E un ensemble de données : $E = \{(x_1, y_1), (x_2, y_2) \ldots (x_n, y_n)\}$
W les paramètres du réseau de neurones initialisés
h la fonction d'activation
η le pas d'apprentissage
repeat
  for all $e_i = (x_i, y_i)$ dans E **do**
    for all j dans $\{1, 2 \ldots k\}$ **do**
      $in = w_j.x_i$
      $err = y_i - h(in)$
      $w_j = w_j + \eta.err.h'(in).x_i$
    end for
  end for
until un certain critère de convergence

Cross-entropy vs. Mean Squared Error

$$H(p, q) = -\sum_x p(x) \log q(x)$$

Modèle num. 1 :

	$\hat{y}$			y			correct ?
e_1	0.3	0.3	0.4	0	0	1	oui
e_2	0.3	0.4	0.3	0	1	0	oui
e_3	0.1	0.2	0.7	1	0	0	non

Cross-entropy vs. Mean Squared Error

$$H(p, q) = -\sum_x p(x) \log q(x)$$

Modèle num. 1 :

	$\hat{y}$			y			correct ?
e_1	0.3	0.3	0.4	0	0	1	oui
e_2	0.3	0.4	0.3	0	1	0	oui
e_3	0.1	0.2	0.7	1	0	0	non

Modèle num. 2 :

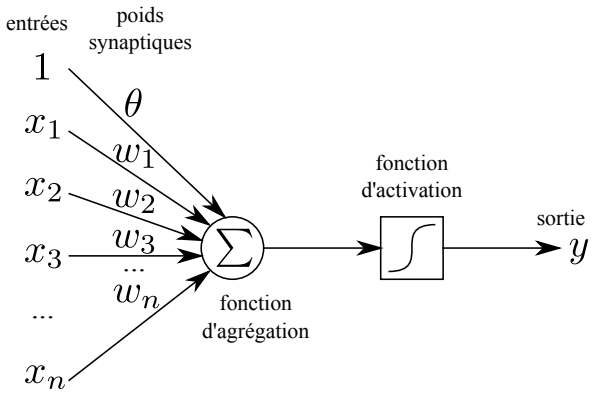
	$\hat{y}$			y			correct ?
e_1	0.1	0.2	0.7	0	0	1	oui
e_2	0.1	0.7	0.2	0	1	0	oui
e_3	0.3	0.4	0.3	1	0	0	non

Perceptron multi-couches - Werbos & Rumelhard (1984-1986)

<u>Sortie du neurone</u>:

$$s = \sum_{i=0}^{n} w_i x_i$$

$$\hat{y} = \frac{1}{1 + e^{-s}}$$

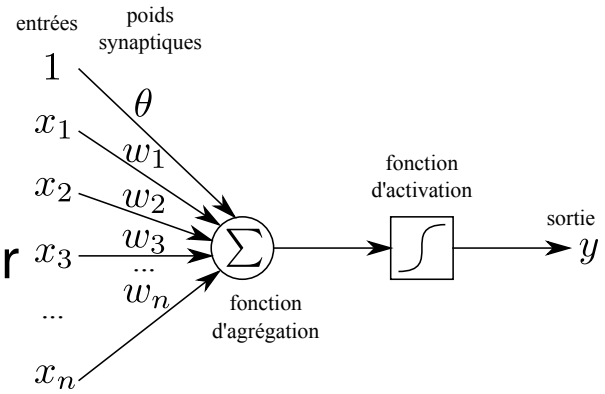


Perceptron multi-couches - Werbos & Rumelhard (1984-1986)

Sortie du neurone:

$$s = \sum_{i=0}^{n} w_i x_i$$

$$\hat{y} = \frac{1}{1+e^{-s}}$$



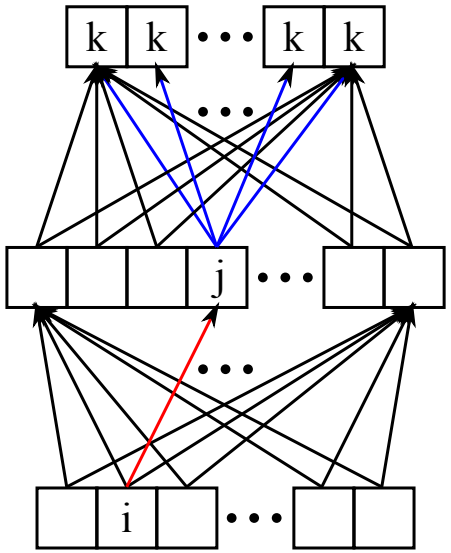
Apprentissage: On veut minimiser l'erreur quadratique $E = \frac{1}{2}\sum_{\mathbf{x}}(y-\hat{y})^2$ par descente de gradient stochastique:

$\Delta \mathbf{w} = \eta \mathbf{x}(y-\hat{y})\hat{y}(1-\hat{y})$

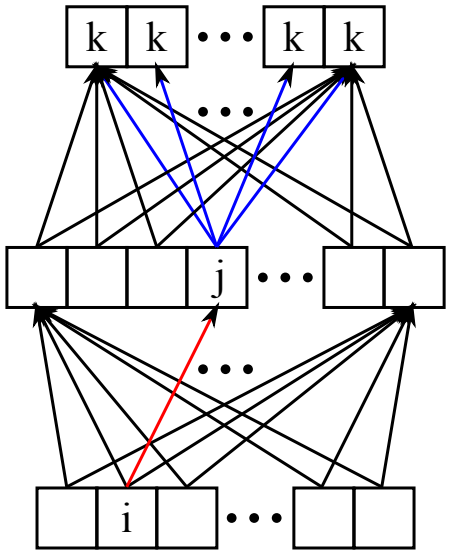
$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} = -\eta \frac{\partial E}{\partial s}\frac{\partial s}{\partial w_i} = \eta \sum_{\mathbf{x}} \frac{\partial \hat{y}}{\partial s}(y-\hat{y})\frac{\partial s}{\partial w_i} = \sum_{\mathbf{x}} \eta \hat{y}(1-\hat{y})(y-\hat{y})x_i$$

Perceptron multi-couches - Werbos & Rumelhard (1984-1986)

Réseau: connectivité complète entre la couche d'entrée, la(les) couche(s) cachée(s) et la couche de sortie



Perceptron multi-couches - Werbos & Rumelhard (1984-1986)

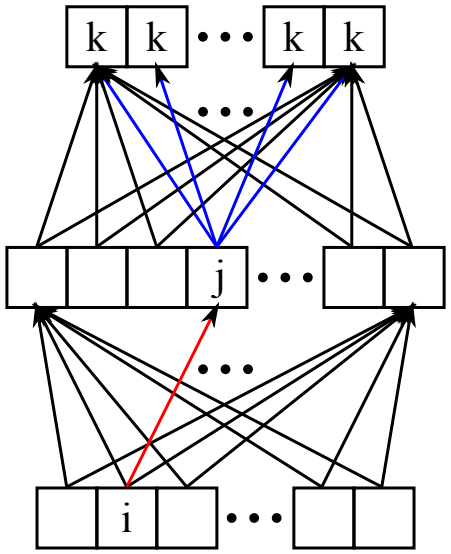


Réseau: connectivité complète entre la couche d'entrée, la(les) couche(s) cachée(s) et la couche de sortie

Apprentissage: On veut minimiser l'erreur quadratique $E = \frac{1}{2}\sum_{\mathbf{x}}(y-\hat{y})^2$ par descente de gradient stochastique, on suppose connus les $\delta_k = -\frac{\partial E}{\partial s_k}$ de la couche supérieure

$$\begin{aligned}
-\frac{\partial E}{\partial s_j} &= -\sum_k \frac{\partial E}{\partial s_k}\frac{\partial s_k}{\partial \hat{y}_j}\frac{\partial \hat{y}_j}{\partial s_j} \\
&= -\sum_k -\delta_k w_{jk}\hat{y}_j(1-\hat{y}_j) \\
&= \hat{y}_j(1-\hat{y}_j)\sum_k \delta_k w_{jk}
\end{aligned}$$

Perceptron multi-couches - Werbos & Rumelhard (1984-1986)



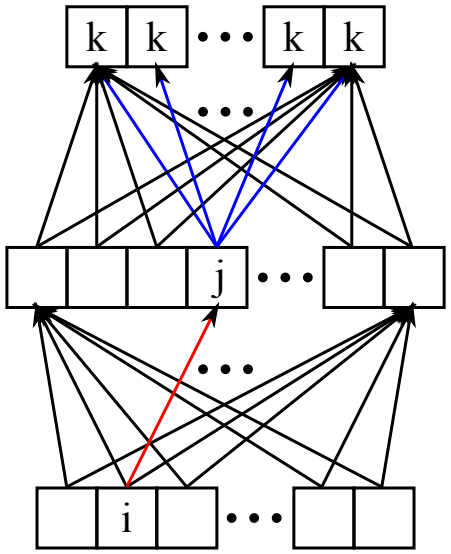
Réseau: connectivité complète entre la couche d'entrée, la(les) couche(s) cachée(s) et la couche de sortie

Apprentissage: On veut minimiser l'erreur quadratique $E = \frac{1}{2}\sum_{\mathbf{x}}(y-\hat{y})^2$ par descente de gradient stochastique, on suppose connus les $\delta_k = -\frac{\partial E}{\partial s_k}$ de la couche supérieure

$$\begin{aligned}
-\frac{\partial E}{\partial s_j} &= -\sum_k \frac{\partial E}{\partial s_k}\frac{\partial s_k}{\partial \hat{y}_j}\frac{\partial \hat{y}_j}{\partial s_j} \\
&= -\sum_k -\delta_k w_{jk}\hat{y}_j(1-\hat{y}_j) \\
&= \hat{y}_j(1-\hat{y}_j)\sum_k \delta_k w_{jk}
\end{aligned}$$

Perceptron multi-couches - Werbos & Rumelhard (1984-1986)

Réseau: connectivité complète entre la couche d'entrée, la(les) couche(s) cachée(s) et la couche de sortie

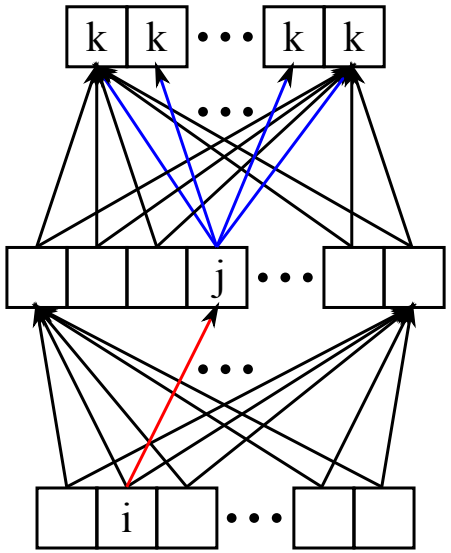


Apprentissage: Pour chaque entrée reçue:

1. Calculer la sortie $\hat{y}$ du réseau par propagation (couche par couche) de l'activité
2. Calculer l'erreur de la couche de sortie: $\delta_k = \hat{y}_k(1 - \hat{y}_k)(y_k - \hat{y}_k)$
3. Rétropropager l'erreur à travers chaque couche j du réseau $\delta_j = \hat{y}_j(1 - \hat{y}_j) \sum_k \delta_k w_{jk}$
4. Modifier chaque poids $\Delta w_{ij} = \eta \delta_j x_i$

Perceptron multi-couches - Werbos & Rumelhard (1984-1986)

Réseau: connectivité complète entre la couche d'entrée, la(les) couche(s) cachée(s) et la couche de sortie



Apprentissage: Pour chaque entrée reçue:

1. Calculer la sortie $\hat{y}$ du réseau par propagation (couche par couche) de l'activité
2. Calculer l'erreur de la couche de sortie: $\delta_k = \hat{y}_k(1 - \hat{y}_k)(y_k - \hat{y}_k)$
3. Rétropropager l'erreur à travers chaque couche j du réseau $\delta_j = \hat{y}_j(1 - \hat{y}_j) \sum_k \delta_k w_{jk}$
4. Modifier chaque poids $\Delta w_{ij} = \eta \delta_j x_i$

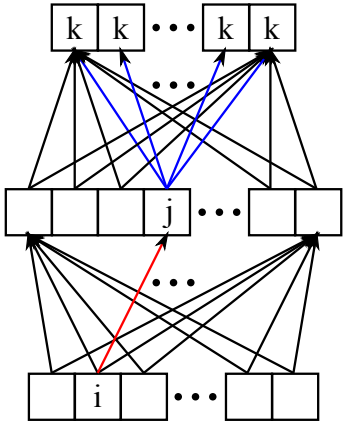
Plan

Approches classiques
- Méthode des K plus proches voisins
- Arbre de classification
- Régression logistique
- Machines à vecteurs supports

Réseaux de neurones artificiels
- Introduction générale
- Le perceptron simple
- Le perceptron multicouches
- Au-delà du MLP

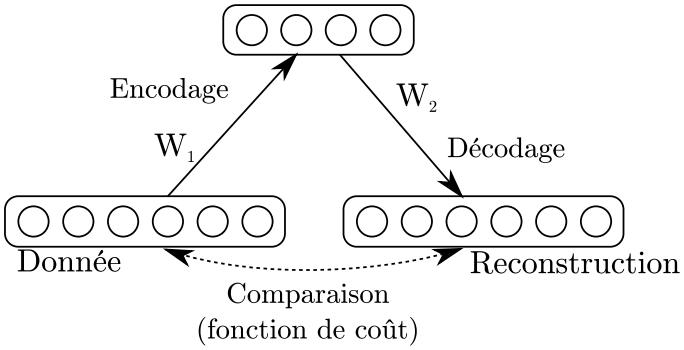
Au-delà du MLP

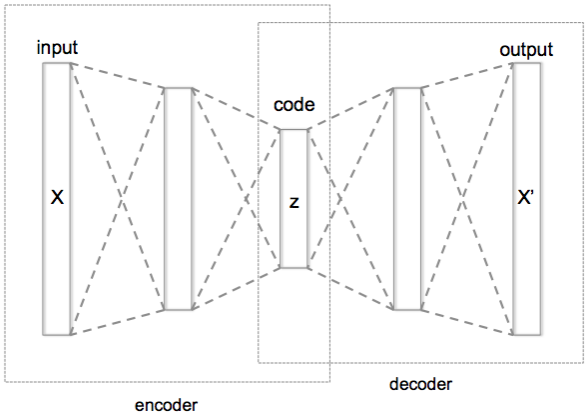
- *Perceptron / Multi-layer perceptron*



Au-delà du MLP

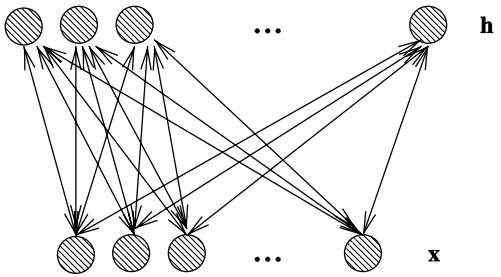
- *Perceptron / Multi-layer perceptron*
- *Auto-encoder / Stacked auto-encoder*



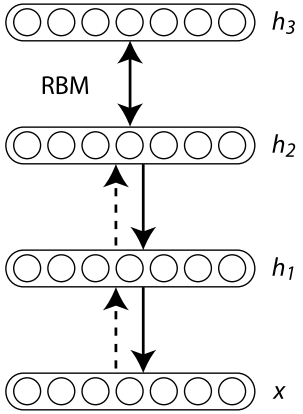


Au-delà du MLP

- *Perceptron / Multi-layer perceptron*
- *Auto-encoder / Stacked auto-encoder*
- *Restricted Boltzmann machine / Deep belief network*

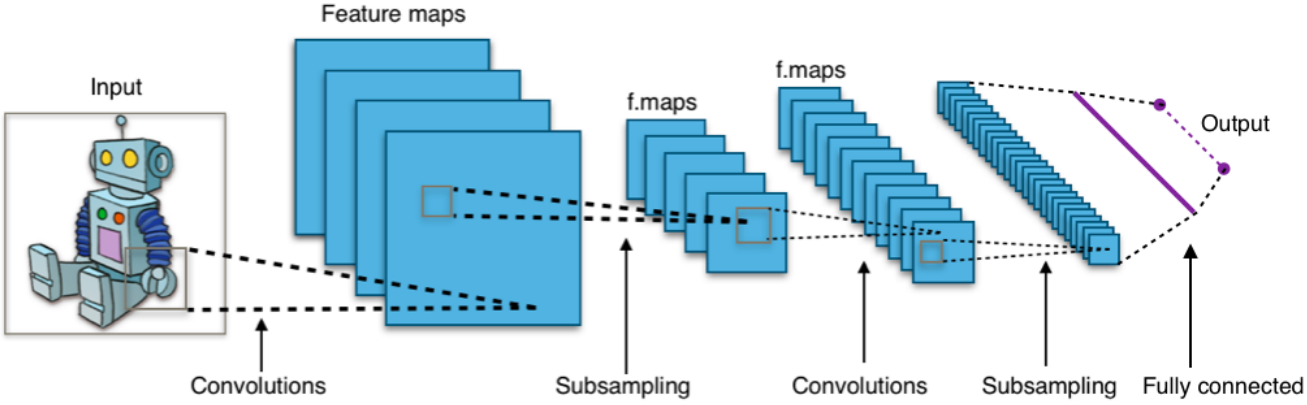


$$P(\mathbf{h}^{\ell-1}, \mathbf{h}^{\ell}) \propto e^{\mathbf{b}'\mathbf{h}^{\ell-1} + \mathbf{c}'\mathbf{h}^{\ell} + \mathbf{h}^{\ell\prime} W \mathbf{h}^{\ell-1}}$$



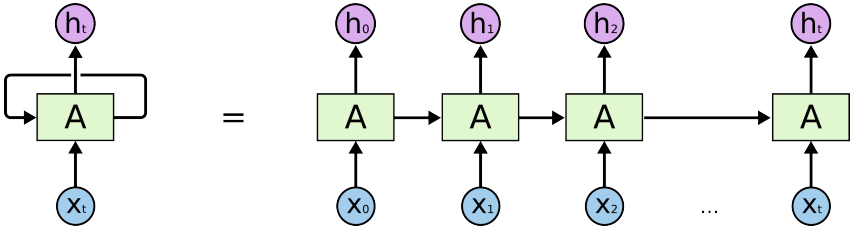
Au-delà du MLP

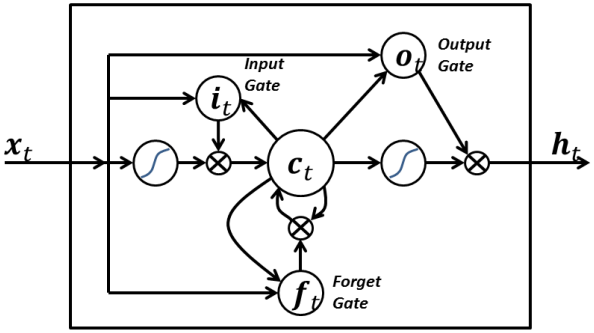
- *Perceptron / Multi-layer perceptron*
- *Auto-encoder / Stacked auto-encoder*
- *Restricted Boltzmann machine / Deep belief network*
- *Convolutional neural network*

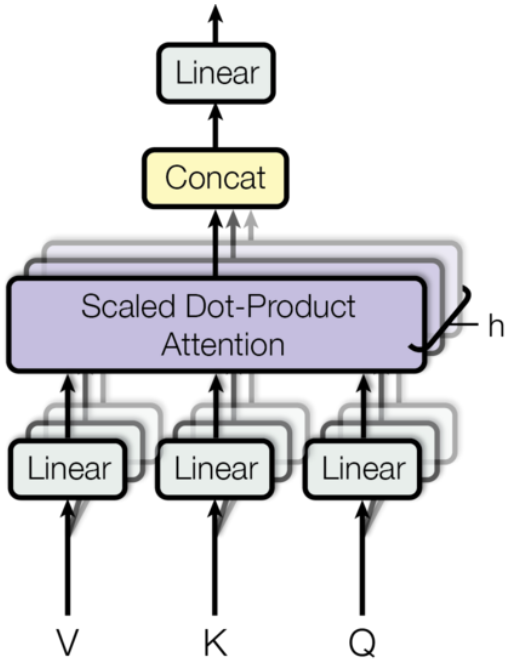


Au-delà du MLP

- *Perceptron / Multi-layer perceptron*
- *Auto-encoder / Stacked auto-encoder*
- *Restricted Boltzmann machine / Deep belief network*
- *Convolutional neural network*
- *Recurrent neural network*





Au-delà du MLP

- *Perceptron / Multi-layer perceptron*
- *Auto-encoder / Stacked auto-encoder*
- *Restricted Boltzmann machine / Deep belief network*
- *Convolutional neural network*
- *Recurrent neural network*
- *Attention models*